

SOFTWARE ENGINEERING

In the context of software development, there are two orthogonal views: **the traditional approach and the object-oriented approach**. These views represent different perspectives on how software is developed and organized.

1. **Traditional Approach:** The traditional approach to software development focuses on functions or procedures. It considers software as a collection of procedures or functions that manipulate isolated data. The main characteristics of the traditional approach include:

- **Emphasis on functions:** The primary focus is on breaking down the system into smaller functions or procedures that perform specific tasks.
- **Sequential process:** The development process follows a sequential order, moving from one phase to another (e.g., requirements gathering, design, implementation, testing) in a linear fashion.
- **Increased complexity:** As the project progresses, the complexity of managing and maintaining the software system tends to increase.
- **Longer duration:** Due to the sequential nature of the process and potential complexities, the duration of the project may be longer.

2. **Object-Oriented Approach:** The object-oriented approach to software development centers on objects, which are combinations of data and functionality. It focuses on organizing software around objects, classes, and modules that can be easily replaced, modified, and reused. The key characteristics of the object-oriented approach include:

- **Emphasis on objects:** The primary focus is on identifying and modeling real-world objects and their interactions within the software system.

- **Modular and reusable:** Objects can be encapsulated into classes, and these classes can be easily reused in different parts of the system or in other projects.
- **Iterative and incremental:** The development process is iterative, with each iteration building upon the previous one. It allows for flexibility and adaptability to changes during development.
- **Reduced complexity and redundancy:** By organizing software around objects, the object-oriented approach aims to reduce complexity and eliminate redundant code or functionality.

SOFTWARE PROCESS

A software process refers to a set of activities that are followed to create a software product. It provides a structured approach to software development and helps ensure that the resulting product is of high quality. There are different types of software processes, but they all involve four fundamental activities :-

1. **Software specification:** This involves defining the functionality and constraints of the software. It is important to clearly understand what the software needs to do and how it should operate.
2. **Software design and implementation:** In this activity, the software is designed and then implemented based on the specifications. It involves creating the actual code and programming the software.
3. **Software validation:** The software is tested to ensure that it meets the requirements and functions as expected. This step is crucial to identify any issues or bugs in the software and ensure its correctness.
4. **Software evolution:** Over time, software needs may change or new features may be required. This activity involves making

updates and modifications to the software to meet changing customer needs.

These activities may have sub-activities and supporting process activities such as requirements validation, documentation, and software configuration management.

In addition to activities, software processes may also include other elements such as products (the outcomes of activities), roles (responsibilities of people involved), and pre- and post-conditions (statements that are true before and after an activity).

Software processes can be categorized as either plan-driven or agile processes. Plan-driven processes involve planning all activities in advance and measuring progress against the plan. Agile processes, on the other hand, involve incremental planning and are more adaptable to changing customer requirements. The choice between these approaches depends on the type of software being developed.

In simpler terms, a software process is a roadmap that guides the development of software. It includes steps like understanding what the software should do, designing and building it, testing it to make sure it works correctly, and adapting it over time as needs change. The process provides stability, control, and organization to ensure the production of quality software. It can be planned in advance or adapted incrementally based on the project's needs.

CHAPTER TWO

REQUIREMENT ELICITATION

Requirement elicitation is a process in software engineering where people who want to create a computer system work together to figure out what the system should do. The people who want the system (we call them clients) and the people who will build the system (we call them developers) talk and discuss to understand the purpose of the system.

During requirement elicitation, clients and developers talk to each other and look at existing

systems if there are any, to find out what the system needs to do. They try to understand what problems the system should solve. Once they have a good understanding of the problems, they create a document called a "requirements specification" that describes what the system should do. This document is like a contract between the clients and the developers.

In simple terms, requirement elicitation is like when you and your friends want to build a cool toy, and you sit together to decide what the toy should be able to do. You talk about what problems the toy should solve and what features it should have. Then, you write down all the things the toy needs to do in a special document that everyone agrees on. That way, everyone knows what the toy will be like and what it should be able to do.

When we talk about requirements specification and analysis model, we are basically talking about two different ways of describing and understanding a computer system that we want to create.

The requirements specification is like a document that uses normal words and sentences to explain what the system should do. It helps us talk and understand the system better with the people who want it, like the clients and the users. It's like when you write a story or a description of something using your own words.

On the other hand, the analysis model is a different way of representing the system. It uses special symbols and diagrams to show how different parts of the system fit together and how they work. It's like drawing a picture or making a map of the system using special signs and shapes.

The important thing to know is that both the requirements specification and the analysis model are trying to show the same information about the system, but they use different languages and ways of expressing it. The requirements specification is more like a conversation, while the analysis model is like a picture or a map. They are both helpful for different purposes. The

requirements specification helps us talk to the people who want the system, while the analysis model helps the developers understand and communicate with each other.

So, imagine you and your friends are planning a big event. You can write a list of things you want to happen in the event using words and sentences. That's like the requirements specification. And then, you can also draw a map or a diagram to show how different parts of the event will be organized, like where the stage will be, where the food will be, and where the games will be. That's like the analysis model. Both the list and the map help you and your friends understand and plan the event, but they use different ways of explaining things.

When we're trying to understand a computer system, we need to know who will use it and what they will do with it. We use two important things to help us describe the system: actors and use cases.

Actors are like the different people, machines, or things that interact with the system. They can be the people who will use the system, other computers that the system needs to talk to, or even the environment where the system will be used. It's like the different characters or players in a game.

Use cases are like stories or scenarios that show how these actors and the system will interact with each other. They describe the different actions or steps that can happen between an actor and the system for a specific part of the system's functionality. It's like playing out different situations or adventures in a game.

For example, imagine you have a game console, and you want to know who will play with it and what they can do. The players who will use the game console are the actors. They can be you, your friends, or even other devices like a controller or a special sensor. The use cases are like the different game levels or challenges you can play. Each use case describes what the player and the game console will do together, like jumping, shooting, or solving puzzles.

So, actors are the different characters or things that interact with the system, and use cases are the fun and exciting things they can do together.

TYPES OR REQUIREMENTS

When we talk about requirements for a computer system, we have different types of requirements that help us understand what the system needs to do.

Functional requirements are like rules or instructions that describe how the system should interact with the things around it, like people, other devices, or the environment. They are independent of how the system will be made or what its purpose is. For example, a functional requirement could be that a game operator should be able to create a new game. It's like saying that the person who runs the game should have the ability to make a new game.

Nonfunctional requirements are about things that are not directly related to how the system works or what it does, but are still important for how well the system performs. They are about aspects like speed, reliability, or user experience. For example, a nonfunctional requirement could be that the system should respond within 1 second. It's like saying that the system needs to react quickly when someone interacts with it.

Constraints or system requirements are specifications or rules that are imposed by the client or the environment. These are specific things that the system must have or follow in order to work with certain hardware or software. For example, a constraint could be that the system must be built using the Java programming language. It's like a special requirement that the client or the environment wants the system to meet.

So, think of it like baking a cake. Functional requirements are like the instructions that tell you how to mix the ingredients and put them in the oven. Nonfunctional requirements are like the things you want the cake to be, like moist, delicious, and not too crumbly. Constraints or system requirements are like special rules you

have to follow, like using a specific type of oven or a certain brand of chocolate.

In summary, functional requirements tell us how the system should interact with its surroundings, nonfunctional requirements focus on things like speed and reliability, and constraints or system requirements are special rules or specifications that need to be followed.

FUNCTIONAL REQUIREMENTS

Functional requirements are like instructions that describe what a software system should do. They explain how the system should interact with its surroundings and what tasks it should perform. Here are some important points about functional requirements:

1. Operations on screens: The requirements should mention what actions can be performed on each screen of the software.
2. Data handling: The requirements should explain how the system will manage and process data.
3. System reports and outputs: The requirements should describe the different types of reports or outputs that the system should generate.
4. Workflows: The requirements should provide information about the different steps or processes that the system will follow.
5. User permissions: The requirements should specify who is allowed to create, modify, or delete data in the system.
6. Regulatory and compliance needs: The requirements should include any requirements related to following rules or regulations.

To create a good functional requirements document, it's important to follow some best practices:

1. Keep requirements separate: Each requirement should focus on a single task

or function to make them clear and specific.

2. Be complete and accurate: Make sure each requirement is fully described and includes all the necessary details.
3. Include technical requirements: Consider any technical specifications or constraints that need to be met.
4. Align with objectives and principles: Link the requirements to the overall goals and principles of successful software delivery.
5. Document critical constraints: If there are any known limitations or restrictions that significantly affect a requirement, they should be clearly mentioned.

Here are some examples of functional requirements:

- The software should automatically validate customer information in the ABC Contact Management System.
- The Sales system should allow users to record customers' sales.
- Only managerial-level employees should have the right to view revenue data.
- The software system should be integrated with a banking API.
- The system should allow users to edit their profile.

When writing requirements, it's important to avoid using confusing terms or phrases such as "support," "but not limited to," "etc.," and "and/or." These terms can make the requirements unclear or ambiguous.

NON FUNCTIONAL REQUIREMENTS

Nonfunctional requirements are like special rules or qualities that a software system should have, but they are not about what the system actually does. They focus on different aspects of the system that make it better or more enjoyable to use. Here are some important points about nonfunctional requirements :-

1. Not directly related to function: Nonfunctional requirements are about

things other than what the system does. They are about how the system performs or how it feels to use.

2. Many different aspects: Nonfunctional requirements cover a wide range of qualities that make the system better. They can be about usability, performance, security, and more.

In software requirement engineering, some common categories of nonfunctional requirements include:

- **Usability:** These requirements focus on making the system easy and enjoyable to use. For example, the system should have a simple and intuitive user interface.
- **Performance:** These requirements deal with how fast and responsive the system is. For example, the system should respond quickly within a certain time limit.
- **Security:** These requirements ensure that the system protects information and prevents unauthorized access. For example, the system should require a strong password for user authentication.
- **Reliability:** These requirements are about how reliable and stable the system is. For example, the system should be able to handle a large number of users without crashing.
- **Scalability:** These requirements address the system's ability to handle an increasing number of users or data. For example, the system should be able to support more users as the business grows.
- **Compatibility:** These requirements ensure that the system works well with different devices or software. For example, the system should be compatible with various web browsers.

So, nonfunctional requirements are like special qualities or rules that make the system better. They cover different aspects like usability,

performance, security, and more. They help ensure that the system is not only functional but also enjoyable and reliable to use.

1) USABILITY

Usability is all about how easy it is for someone to learn, use, and understand a system or a part of it. It's like how simple and straightforward something is to operate.

Usability requirements are the rules or guidelines that make sure the system is easy to use. For example, the requirements might say that the system should follow certain design conventions, like using specific colors, logos, and fonts. This helps make the system look and feel familiar and consistent to the users.

Usability requirements also include things like how much help is available to the user while they're using the system. This can include online guides or manuals that explain how to use the system or how to do certain tasks.

So, imagine you have a new gadget or a game console. Usability is about how easy it is for you to figure out how to use it without getting confused. The usability requirements might say that the buttons should be labeled clearly, the menus should have pictures or icons to help you understand them, and there should be a user manual or online help available to answer your questions.

In simpler terms, usability is about how user-friendly something is and usability requirements are the rules that make sure things are easy to understand and use.

2) RELIABILITY

Reliability is about how well a system or a part of it can do its job consistently and without any problems. It's like how trustworthy and dependable something is.

Reliability requirements are the rules or conditions that ensure the system works reliably. They include things like how often the system should have problems or fail, and how well it can handle certain faults or security attacks.

These requirements help make sure the system can be trusted to perform its functions when needed.

Sometimes, the term "dependability" is used instead of reliability. Dependability means that you can rely on the system to provide the service it's supposed to deliver. It includes not just reliability, but also other qualities like robustness and safety.

Robustness is about how well the system can handle difficult or challenging situations. For example, it's about how well a video game can continue running even if there are some errors or if the environment conditions are not perfect.

Safety is about making sure the system doesn't cause any harmful or catastrophic consequences to the surroundings. It's like how a car should be designed to be safe and not cause accidents or harm people.

So, imagine you have a computer or a smartphone. Reliability is about how often it works without crashing or having errors. Reliability requirements might say that the system should not fail more than a certain number of times within a specific period. Dependability is about being able to trust that the computer or smartphone will work when you need it. It includes not just reliability, but also how well it can handle difficult situations and how safe it is to use.

In simpler terms, reliability is about how trustworthy and consistent a system is, and reliability requirements are the rules that make sure the system works well. Dependability includes reliability, robustness, and safety, and it's about being able to rely on the system to do its job effectively.

3) PERFORMANCE

Performance requirements are about how fast and efficient a system or a part of it can work. They focus on measurable aspects that tell us how well the system performs certain tasks.

Response time is about how quickly the system reacts or responds when someone gives it an

input or a command. It's like how fast the system can start doing what you ask it to do.

Throughput is about how much work or tasks the system can handle and complete within a specific amount of time. It's like how many things the system can do in a given period.

Availability is about how often the system is operational and accessible for use. It's like how frequently the system is ready and working when you need it.

Accuracy is about how precise and correct the system's output or results are. It's like how well the system can give you the right answers or information.

So, let's imagine you have a computer game. Performance requirements are about how fast the game reacts when you press a button or make a move. The response time requirement might say that the game should respond within a certain number of milliseconds after you press a button.

Throughput is about how many actions or tasks the game can handle at the same time. The throughput requirement might say that the game should be able to process a certain number of moves or actions per second.

Availability is about how often the game is working and accessible. The availability requirement might say that the game should be available to play at least 99% of the time, meaning it can only be down or unavailable for a very small percentage of the time.

Accuracy is about how well the game provides accurate results or scores. The accuracy requirement might say that the game's scoring system should be precise and give the correct points for each action or achievement.

In simpler terms, performance requirements are about how fast, how much, how often, and how accurate a system can perform its tasks. They help ensure that the system works efficiently and effectively.

4) SUPPORTABILITY

Supportability requirements are about how easy it is to make changes to a system after it has been deployed or put into use. They focus on making the system adaptable, maintainable, and capable of working in different environments.

Adaptability is about how easily the system can be changed or adjusted to handle new concepts or features related to the specific area it's being used in. It's like how well the system can learn or accommodate new things as needed.

Maintainability is about how easy it is to make changes to the system to fix any problems or to add new technology. It's like how simple it is to update or improve the system without breaking anything.

Internationalization is about how well the system can be changed to work with different international conventions. This includes things like different languages, units of measurement, and number formats. It's like how the system can be made compatible and useful for people in different countries or regions.

Portability is about how easily the system can be transferred or moved from one hardware or software environment to another. It's like how well the system can work on different devices or platforms without needing a lot of changes or adjustments.

So, imagine you have a smartphone app. Supportability requirements are about how easy it is to update the app with new features or fix any issues. Adaptability means the app can handle new concepts or functionalities as the needs of users change. Maintainability means the app can be updated with new technology without causing problems. Internationalization means the app can work with different languages and number formats, making it useful for people around the world. Portability means the app can work on different types of phones or operating systems without needing major changes.

In simpler terms, supportability requirements are about how easy it is to make changes to the system and keep it up-to-date. They include adaptability, maintainability, internationalization,

and portability, which are all about making the system flexible, easy to fix, usable in different regions, and compatible with different devices.

REQUIREMENT ELICITATION ACTIVITIES

Requirements elicitation activities are a set of steps or tasks that help us understand a problem and define what a computer system needs to do. These activities use a modeling technique called UML (Unified Modeling Language) to visualize and describe the system's specifications.

The process begins by identifying the different actors, which are the people, machines, or things that will interact with the system. Next, we identify various scenarios or situations where the system will be used, considering the actions and interactions of the actors. Use cases are then created, which represent specific tasks or functions that the actors and the system will perform together.

To make the use cases more detailed, they are refined by adding more information about the steps, inputs, and outputs involved in each case. Relationships among the actors and use cases are identified to understand how they are connected or related to each other.

Additionally, nonfunctional requirements are considered, which include aspects like system performance, security, and usability. These requirements are essential for ensuring the overall quality of the system.

By following these elicitation activities, we can create a clear and comprehensive description of what the system should do and how it should behave. This helps in effectively planning and developing the computer system to meet the needs and expectations of the users and stakeholders.

Requirements elicitation activities help us understand a problem and turn it into a set of specifications for a computer system. This is done using a method called modeling with UML (Unified Modeling Language), which helps us visualize and describe the system.

Here are the steps involved in requirements elicitation activities:

1. **Identifying Actors:** We figure out the different people, machines, or things that will interact with the system. They are called actors. For example, in a shopping app, actors could be the customers, the sellers, and the payment gateway.
2. **Identifying Scenarios:** We think about different situations or scenarios where the system will be used. We consider what the actors will do and how they will interact with the system. For example, a scenario could be a customer searching for a product and making a purchase.
3. **Identifying Use Cases:** Use cases are like stories or descriptions of what the system will do in each scenario. They represent specific tasks or functions that the actors and the system will perform together. For example, a use case could be "Customer places an order."
4. **Refining Use Cases:** We take the initial use cases and make them more detailed and specific. We add more information about the steps, inputs, and outputs involved in each use case. This helps us understand exactly what needs to happen in the system.
5. **Identifying Relationships Among Actors and Use Cases:** We look at how the actors and use cases are connected or related to each other. This helps us understand the flow of interactions in the system. For example, a customer actor may be related to multiple use cases like searching, adding items to the cart, and making payments.
6. **Identifying Nonfunctional Requirements:** Apart from the specific tasks, we also consider other requirements like how fast the system should respond, how secure it needs to be, or how easy it should be to use. These are called nonfunctional

requirements, and they are important for the overall quality of the system.

So, imagine you and your friends are planning a board game. The requirements elicitation activities would involve figuring out who will play the game (actors), thinking about different situations during the game (scenarios), describing what each player will do (use cases), making the use cases more detailed, understanding how players and tasks are connected, and considering any additional requirements like the game's speed or safety.

In simpler terms, requirements elicitation activities are about understanding a problem and creating a description of what the computer system needs to do. We use different techniques, like identifying actors, scenarios, and use cases, to map out the requirements. It's like making a plan for how the system will work and what it will be able to do.

IDENTIFYING ACTORS

In the process of requirements elicitation, one of the key steps is identifying the actors. Actors are the different types of users and external systems that interact with the computer system being developed. By understanding these actors, we can better determine the specific functionality and tasks that the system needs to support. This involves asking questions about user groups, their main functions, secondary functions, and any external systems the system needs to interact with. Once the actors are identified, the next step is to define the functionality accessible to each actor through scenarios and use cases. This helps create a clear understanding of how the system will be used and what it needs to accomplish for different users and external entities.

Identifying actors involves figuring out the different types of users and external systems that will interact with the computer system we're building. We ask questions like:

1. Which user groups will use the system for their work?
2. Which user groups will perform the main functions of the system?

3. Which user groups will have secondary functions like maintenance and administration?
4. Are there any external hardware or software systems that the system needs to interact with?

By answering these questions, developers can understand the various users that the system will support. Once the actors are identified, the next step is to determine what each actor should be able to do in the system. This is done by using scenarios, which are like stories that describe specific situations, and formalizing them into use cases, which are detailed descriptions of tasks or functions.

In simpler terms, identifying actors means figuring out the different types of users and external systems that will be involved with the computer system. We ask questions to understand who will use the system and what they will do. This helps us determine the specific tasks or functions that each user should be able to perform.

IDENTIFYING SCENARIOS

In the process of requirements elicitation, another important activity is identifying scenarios. Scenarios are detailed descriptions of specific situations or tasks that users will encounter when using the computer system or application. They provide a narrative of what people do and experience as they interact with the system.

During this activity, developers closely observe users and create a set of scenarios that outline the typical functionality the future system should provide. These scenarios serve as a means for developers to communicate with users and gain a deeper understanding of the application domain, or the specific area or context in which the system will be used.

A scenario is a focused and informal description that highlights a single feature or task of the system, as seen from the perspective of a single user or actor. It provides a concrete example of how the system will be used and what actions or interactions are expected.

In simpler terms, identifying scenarios involves creating detailed descriptions of specific situations or tasks that users will encounter when using the system. Developers observe users to understand their needs and then write scenarios that help them better understand how the system should function in real-life situations. These scenarios provide a clear picture of what the system needs to do from the user's point of view.

To identify scenarios, we can ask the following questions :-

1. What tasks does the user want the system to do? This helps us understand the specific actions or functions the user expects from the system.
2. What information does the user need to access? We explore what kind of data the user wants to see and interact with. We also determine who creates this data and if it can be changed or removed, and by whom.
3. Are there any external changes that the user needs to inform the system about? We consider if there are any updates or events outside the system that the user needs to communicate to the system, and how frequently this should be done.
4. Which events does the system need to inform the user about, and how quickly should this information be delivered? We determine if there are any important updates or notifications that the system needs to provide to the user, and how fast this should happen.

In simpler terms, when identifying scenarios, we ask questions to understand what tasks the user wants the system to do and what information they need. We also consider any external changes or events that the user or the system should be aware of. By answering these questions, we can create detailed descriptions of specific situations or tasks that users will encounter when using the system, providing a better understanding of how the system should function in different scenarios.

IDENTIFYING USE CASES

A scenario is like a specific example or situation that shows how a particular function or task in the system can be used. It's like a real-life story that describes what happens step by step when someone interacts with the system.

A use case, on the other hand, is like a collection of all possible scenarios related to a specific function or task. It's like a summary of different situations that can occur when using that particular function or task in the system.

When developers and users agree on a set of scenarios, developers can create use cases based on those scenarios. The use cases help us understand the overall scope or coverage of the system and provide a complete flow of events that happen when the use case is initiated or started.

In simpler terms, a scenario is an example that shows how a specific function or task works in the system. A use case is a summary that includes all the possible scenarios related to that function or task. Use cases help us understand how different interactions and events flow through the system when using that function or task.

SIMPLE USE CASE WRITING GUIDE

When creating use cases, there are certain guidelines to follow:

1. **Use a verb phrase to name the use case:** The name of the use case should indicate what the user is trying to accomplish. It should be clear and descriptive.
2. **Name actors with noun phrases:** Actors, which are the users or external systems, should be named using nouns that describe their roles or responsibilities in the system.
3. **Clearly define the system boundary:** It should be clear which steps are performed by the user (actor) and which steps are performed by the system. This helps

distinguish the interactions between the user and the system.

4. **Use active voice for step descriptions:** Use case steps should be written in the active voice to explicitly state who is accomplishing each step. This makes it clear and easy to understand.
5. **Clearly show the causal relationship between steps:** The relationship between each step should be clear, showing the cause and effect or the logical flow between them.
6. **Describe a complete user transaction:** A use case should describe a complete sequence of interactions between the user and the system to achieve a specific goal or task.
7. **Document exceptions separately:** Any exceptional or error situations should be described separately from the main flow of events.
8. **Avoid describing the user interface:** A use case should focus on the user's actions and the system's responses, rather than specific details of the user interface design.
9. **Keep use cases concise:** Use cases should not be too long. Ideally, they should be limited to two or three pages. If a use case becomes lengthy, it can be decomposed into smaller use cases using relationships like "include" and "extend".

In simpler terms, when creating use cases, we give them names that describe what the user wants to accomplish. Actors are named based on their roles. We clearly define which steps are done by the user and which are done by the system. Use case steps are described in a way that shows who is doing each action. The relationship between steps should be clear, and the use case should describe a complete user transaction. Any exceptions or errors are described separately. The focus should be on the user's actions and the system's responses, not the

specific user interface. Use cases should be kept concise and, if needed, can be divided into smaller parts.

REFINING USE CASES

Refining use cases is an activity where developers add more details to the use cases to make sure the requirements specification is complete. They describe each use case more thoroughly and consider how the system should behave in the presence of errors or exceptional conditions.

In simpler terms, refining use cases means developers take the existing use cases and provide more information about them. They make sure all the necessary details are included and think about how the system should handle errors or unusual situations. This helps ensure that the requirements for the system are fully described and well-defined

IDENTIFYING RELATIONSHIPS AMONG USE CASES

Identifying relationships among use cases is an activity where developers determine how different use cases are related to each other. They look for dependencies or connections between use cases and consolidate the use case model by extracting common functionalities. This ensures that the requirements specification is consistent and well-organized.

In simpler terms, identifying relationships among use cases means that developers look at how different tasks or functions in the system are connected. They find out which use cases rely on or depend on other use cases to be completed. They also identify common functionalities that are shared by multiple use cases and separate them to avoid repetition. This helps ensure that the requirements for the system are clear, coherent, and don't have any conflicts or contradictions

IDENTIFYING NON-FUNCTIONAL REQUIREMENTS

Identifying nonfunctional requirements is an activity where developers, users, and clients come together to agree on certain aspects of the system that are not directly related to its functionality. These aspects include constraints or specifications

on how the system should perform, how it should be documented, the resources it should consume, its security measures, and its overall quality.

Let's break it down in simpler terms and go into more detail:

1. **Performance:** This refers to how well the system should perform in terms of speed, responsiveness, and efficiency. It includes requirements like response time (how quickly the system should react), throughput (how much work the system can handle), and availability (how often the system should be accessible).
2. **Documentation:** This involves requirements related to providing clear and comprehensive documentation for the system. It ensures that users and developers have access to detailed instructions, user manuals, guides, or any other necessary documentation.
3. **Resources:** These requirements focus on how the system should utilize resources such as memory, processing power, network bandwidth, or storage space. It may include limitations or specifications on resource usage to ensure the system operates efficiently.
4. **Security:** This refers to requirements related to protecting the system and its data from unauthorized access, breaches, or malicious activities. It involves measures like user authentication, data encryption, access control, and adherence to relevant security standards.
5. **Quality, Reliability, and Supportability:** These requirements encompass various aspects related to the overall quality, reliability, and supportability of the system. It includes factors like robustness (how well the system handles errors or unexpected situations), maintainability (how easily the system can be maintained or modified), and support options (availability of technical support or help desk services).

6. **Interface:** This involves requirements concerning the user interface and how it should look, feel, and function. It includes aspects like usability (how user-friendly and intuitive the interface should be) and accessibility (ensuring the interface is usable for individuals with disabilities).
7. **Legal and Compliance:** These requirements address legal and regulatory obligations that the system must adhere to. It may include data privacy regulations, industry-specific standards, intellectual property rights, or any other legal considerations relevant to the system's domain.

In summary, identifying nonfunctional requirements is about defining specifications beyond the system's functionality. It includes aspects such as performance, documentation, resource usage, security, quality, interface, and legal compliance. These requirements ensure that the system not only performs its tasks but also meets certain standards of performance, usability, security, and compliance with relevant regulations.

REQUIREMENT VALIDATION (COMPLETENESS , CONSISTENCY , CLARITY AND CORRECTNESS)

Requirement validation is an important step in the development process where the client, users, and developers continuously check and confirm that the requirements specification is complete, consistent, clear, and correct. It ensures that everyone involved has a shared understanding of what the system needs to do.

Let's break it down in simpler terms and provide more detail:

1. **Completeness:** The requirements specification is considered complete if it includes all possible scenarios and aspects of the system. It means that all the different situations and behaviors the system may encounter are described, including exceptional cases.
2. **Consistency:** Consistency means that the requirements specification does not

contradict itself. There should be no conflicting or contradictory statements within the document. It ensures that all requirements align and work together to achieve the desired system behavior.

3. **Clarity:** Clarity refers to the unambiguous and clear expression of requirements. Each requirement should have only one interpretation and should be easily understandable by all stakeholders. Ambiguous terms, words with multiple meanings, should be avoided to prevent confusion and misunderstandings.
4. **Correctness:** Correctness means that the requirements accurately represent the system that the client needs and that the developers intend to build. The requirements should describe the desired features and behavior of the system and its environment of interest, without including unintended or unnecessary features.
5. **Ambiguous terms:** Ambiguous terms are words that can be interpreted in different ways and can lead to misunderstandings. Examples of such terms include "minimize," "maximize," "rapid," "easy," "sufficient," "adequate," and "quick." To ensure clarity, it's important to provide clear definitions or explanations for such terms.

Requirement validation is crucial because it helps ensure that everyone is on the same page and that the requirements accurately capture what the system should do. It prevents misunderstandings, inconsistencies, and misinterpretations that could lead to costly errors during development.

In summary, requirement validation is about checking that the requirements specification is complete, consistent, clear, and correct. It confirms that all necessary scenarios and behaviors are described, there are no contradictions, the requirements are unambiguous, and they accurately represent the desired system. This validation process is essential for successful

development and ensuring that the system meets the client's needs and expectations.

REQUIREMENT-ELICITATION METHODS/ TECHNIQUES

During the process of requirements elicitation, various methods can be used to gather information and extract requirements from stakeholders. Here are some commonly used methods:

1. **Interviews:** Interviews involve direct conversations between the analysts and stakeholders to gather information and understand their perspectives. It allows for in-depth discussions, clarifications, and exploration of requirements.
2. **Document analysis:** This method involves studying existing documents such as user manuals, business processes, and technical specifications to extract relevant information and requirements. It helps in understanding the current system and its documentation.
3. **Brainstorming-sessions:** Brainstorming involves bringing together a group of stakeholders to generate ideas, suggestions, and requirements collaboratively. It encourages creative thinking, open discussion, and the exploration of different perspectives.
4. **Facilitated Application Specification Technique (FAST):** FAST is a workshop-based technique that uses visual models and structured discussions to elicit requirements. It involves stakeholders actively participating in creating models and specifying requirements.
5. **Quality Function Deployment (QFD):** QFD is a method that focuses on translating customer needs into specific technical requirements. It uses matrices and customer-driven prioritization techniques to align requirements with customer expectations.

6. **Use Case Approach:** The use case approach involves identifying actors, scenarios, and use cases to describe the interactions and functionality of the system. It helps in understanding user requirements and specifying system behavior.

The success of an elicitation technique depends on factors such as the experience and skills of the analysts, developers, users, and customers involved. The maturity of these individuals influences their ability to effectively communicate, understand needs, and extract requirements using the chosen elicitation technique.

In summary, requirements elicitation can involve various methods such as interviews, document analysis, brainstorming, FAST, QFD, and the use case approach. Each method offers a unique way of gathering information and extracting requirements from stakeholders. The effectiveness of these methods depends on the skills and experience of the individuals involved in the process.

1) INTERVIEWS

The objective of conducting an interview is to understand what the customer expects from the software. Since it's not possible to interview every person involved, representatives from different groups are selected based on their knowledge and credibility.

Interviews can be either open-ended or structured. In open-ended interviews, there is no fixed agenda, and the interviewer asks questions to understand the problem or requirements. In structured interviews, the interviewer has a set of questions prepared in advance to guide the discussion. Sometimes, a questionnaire is used to ensure important points are covered during the interview.

In simpler terms, an interview helps us learn what the customer wants from the software. We can't talk to everyone, so we choose knowledgeable people to represent different groups. Interviews can be more flexible or follow

a structured approach with prepared questions or questionnaires. The goal is to gather information and understand the customer's expectations.

2) DOCUMENT ANALYSIS

Document analysis is a process where we gather and review all the existing documents that are relevant to our business goal or may contain information related to a potential solution. These documents can include various types of written materials such as business plans, market studies, contracts, requests for proposal, statements of work, memos, guidelines, procedures, training guides, and more.

By analyzing these documents, we aim to extract valuable information and insights that can help us understand the current situation, identify problems or opportunities, and gather requirements for a new solution. The documents may also include feedback from customers, reports of issues or suggestions, and specifications of existing systems or products.

In simpler terms, document analysis means going through all the written materials that are important for our project or problem. We look at things like plans, studies, contracts, reports, guidelines, and more. By studying these documents, we can gain valuable information that helps us understand the situation and gather requirements for a potential solution.

3) BRAIN STORMING SESSIONS

Brainstorming sessions are group activities where people come together to generate a lot of new ideas and share their viewpoints. The main goal is to encourage creative thinking and gather a wide range of perspectives.

In these sessions, it's important to have a skilled facilitator who can manage any biases or conflicts that may arise within the group. The facilitator ensures that everyone's ideas are heard and valued.

During the brainstorming session, all ideas are documented so that everyone can see them. This helps in keeping track of all the suggestions and ensures that no idea is overlooked.

At the end of the session, a document is prepared that lists all the requirements that were discussed. If possible, the requirements are prioritized to identify the most important ones.

In simpler terms, brainstorming sessions are group activities where people share their ideas and viewpoints. A facilitator helps guide the session and ensure everyone's ideas are heard. All ideas are written down, and at the end, a document is created listing the requirements and their priority, if possible. The aim is to generate a lot of new ideas and gather a wide range of perspectives.

4) FACILITATED APPLICATION SPECIFICATION TECHNIQUE

FAST, or Facilitated Application Specification Technique, is a requirements gathering method that aims to bridge the gap between the expectations of developers and customers. It is a team-oriented approach where participants collaborate to gather and define requirements for a software application or system.

In a FAST session, attendees are asked to individually create lists of three types of items:

1. Things that are part of the environment surrounding the system.
2. Things that the system produces or outputs.
3. Things that the system uses or takes as inputs.

After creating their lists, the participants combine them and eliminate any duplicate entries. The team then splits into smaller sub-teams, each focusing on developing detailed specifications for different items from the consolidated list. By working collaboratively, the sub-teams create mini-specifications that capture the requirements for each item.

Finally, all the inputs from the meeting, including the consolidated list and the sub-teams' mini-specifications, are used to draft a comprehensive set of specifications for the system. This approach helps ensure that the expectations of both

developers and customers are considered and included in the requirements specification.

In summary, FAST is a method that involves collaborative teamwork to gather and define requirements. Participants create lists of environmental factors, system outputs, and system inputs. These lists are consolidated, and sub-teams develop detailed specifications for different items. The outputs from the meeting are then used to create a comprehensive set of specifications for the system.

5) QUALITY FUNCTION DEPLOYMENT

In this technique, the main focus is on making the customer happy, so it emphasizes requirements that are valuable to the customer. There are three types of requirements identified:

1. **Normal requirements:** These are the basic objectives and goals of the proposed software that are discussed with the customer. For example, for a result management system, normal requirements could include entering marks and calculating results.
2. **Expected requirements:** These requirements are so obvious that the customer doesn't need to explicitly mention them. An example would be protecting the software from unauthorized access.
3. **Exciting requirements:** These are features that go beyond the customer's expectations and provide great satisfaction when they are present. For instance, if unauthorized access is detected, the system should automatically create a backup and shut down all processes.

Here are the major steps involved in this procedure:

1. **Identify all the stakeholders:** This means listing out all the people involved in the project, such as users, developers, and customers.

2. **List out all requirements from the customer:** Gather all the things that the customer wants the software to do.

3. **Assign importance to each requirement:** Give a value or rating to show how important each requirement is.

4. **Categorize the final list of requirements:** At the end, classify each requirement into one of the following categories:

- Possible to achieve: These are requirements that can be implemented.
- Should be deferred and the reason for it: Some requirements may need to be postponed for later due to certain reasons.
- Impossible to achieve and should be dropped off: Some requirements may not be possible to fulfill, so they should be removed from the list.

In simpler terms, this technique focuses on making the customer happy by considering their important requirements. There are three types of requirements: normal, expected, and exciting. The steps involve identifying stakeholders, listing customer requirements, assigning importance to each requirement, and categorizing them as achievable, deferred, or impossible to achieve.

6) USE CASE APPROACH

This technique combines words and pictures to help us understand the requirements better. A scenario is an example of how a specific function or task is carried out in the system.

Use cases describe the "what" of a system, not the "how." They provide a functional view of the system by describing the interactions between actors (external agents like people or machines) and the system. Use cases show who does what with the system. A complete set of use cases covers all the possible ways the system can be used.

The use case design has three main components:

1. **Actors:** Actors are the external agents that interact with the system. They can be people, machines, or other entities. Actors are represented as stick figures. There are two types of actors: primary actors, who need assistance from the system to achieve their goals, and secondary actors, who provide assistance to the system.
2. **Use cases:** Use cases describe the sequence of interactions between actors and the system. They show what actions actors take and how they interact with the system. Use cases capture different functionalities or tasks the system can perform.
3. **Use case diagram:** A use case diagram is a graphical representation of what happens when an actor interacts with the system. It shows the functional aspects of the system by illustrating the relationships between actors and use cases.

In simpler terms, this technique uses pictures and words to understand the system's requirements. A scenario is an example of how something is done in the system. Use cases describe what the system does and how actors interact with it. Actors are the external agents, like people or machines, and they can be primary or secondary. Use case diagrams show the interactions between actors and the system in a visual way.

Software Development Life Cycle Models

A software development life cycle (SDLC) model is a framework that describes the stages involved in the development of software , from its conception to its deployment and maintenance. It is a process that software development teams used to guide their work and ensure that software products are delivered on time , with in budget and to the required level of quality.

There are several different SDLC models that have been developed over the years , each with its

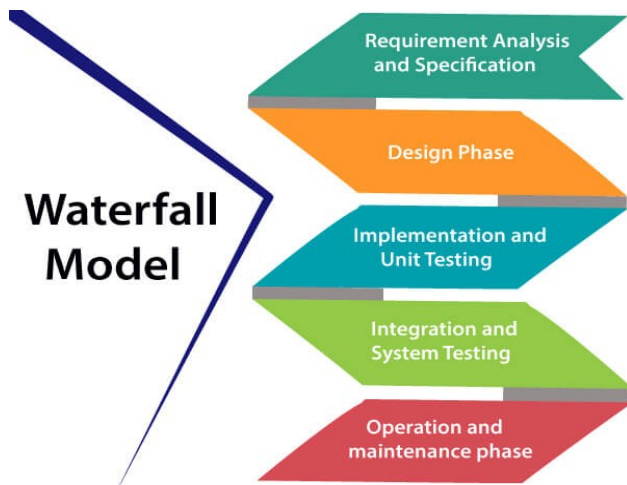
own set of advantages and disadvantages , some of the most common SDLC models include :-

- Waterfall model
- Agile model
- spiral model
- v-model
- incremental model
- iterative model

Waterfall model

The waterfall model is a sequential design process used in software development that follows a linear and a rigid approach. It consists of a series of phases that must be completed in a linear sequence , with each phase dependent on the previous one , before moving on to the next. These phases typically include requirements gathering and analysis , design implementation , testing , and maintenance. Once a phase is completed , the process moves on to the next and there is no turning back or overlapping of phases. This model assumes that all the requirements are known and fixed at the beginning of the project , and changes are not expected or are very minimal. The waterfall model is often criticized for its inflexibility and lack of adaptability to changes in requirements.

The term “waterfall” in the context of software development refers to the sequential flow of the development process , where each phase must be completed before moving on to the next. The name comes from the analogy of a water fall , where the water flows downwards in a series of steps or stages. Similarly , in the waterfall model , each phase or stage of the development process flows down to the next phase and it is not possible to move back up once a phase is completed.



The waterfall model is a software development model that follows a sequential approach with the following steps :-

- 1) **Requirements Gathering and Analysis** :- In this phase , the requirements for the software are gathered and analyzed to determine what the software needs to do and what features are needed.
- 2) **Design** :- In this phase , the software architecture and design are created based on the requirements gathered in the previous phase.
- 3) **Implementation** :- This phase involves the actual coding of the software based on the design created in the previous phase.
- 4) **Testing** :- In this phase , the software is tested to ensure it meets the requirements and is free of errors or bugs.
- 5) **Deployment** :- Once the software has been tested and approved , it Is deployed to the production environment.
- 6) **Maintenance** :- This phase involves ongoing maintenance and support of the software including bug fixes , updates and enhancements.

When to use SDLC Waterfall model ?

Some circumstances where the used of the waterfall model is most suited are :-

- When the requirements are constant and not changed regularly
- A project is short
- The situation is calm

- Where the tools and technology used is consistent and is not changing
- When resources are well prepared and are available to use.

Advantages of Waterfall model

- ✓ This model is simple to implement also the number of resources that are required for it is minimal
- ✓ The requirements are simple and explicitly declared; they remain unchanged during the entire project
- ✓ The start and end points for each phase is fixed , which make it easy to cover progress
- ✓ The release date for the complete product , as well as its final cost , can be determined before development.
- ✓ It gives easy to control and clarity for the customer due to a strict reporting system.

Disadvantages of waterfall model

- ✓ In this model , the risk factor is higher , so this model is not suitable for more significant and complex projects
- ✓ This model can no accept the changes in requirements during development.
- ✓ It becomes tough to go back to the phase , for example , if the application has now shifted to the coding phase , and there is a change in requirement , it becomes tough to go back and change it.
- ✓ Since the testing done at a larger stage , it does not allow identifying the challenges and risks in the earlier phase , and there is a change in requirement , it becomes tough to go back and change it.
- ✓ Since the testing done at a larger stage , it does not allow identifying the challenges and risks in the earlier phase , so the risk reduction strategy is difficult to prepare.

Iterative Model

The iterative model is a software development model where software is developed in small chunks or iterations, each of which includes all the phases of the software development life cycle, such as planning, design, implementation and testing. In contrast to the waterfall model, which is linear and sequential, the iterative model is cyclical and allows for feedback and adjustments throughout the development process. Each iteration results in a working software product that can be tested and evaluated providing valuable insights that can inform the next iteration, this approach is often used in Agile development methodologies.

The point of having iterations in the iterative model is to allow for continuous feedback and improvement throughout the development process. Each iteration involves a cycle of planning, designing, implementing, testing and evaluating. By breaking down the project into smaller parts, teams can focus on improving and refining each iteration before moving on to the next one.

This allows for more flexibility and adaptability as changes can be made at each iteration based on feedback and testing. Iterative development also helps to reduce risk as potential issues or defects can be identified and addressed early on in the process.

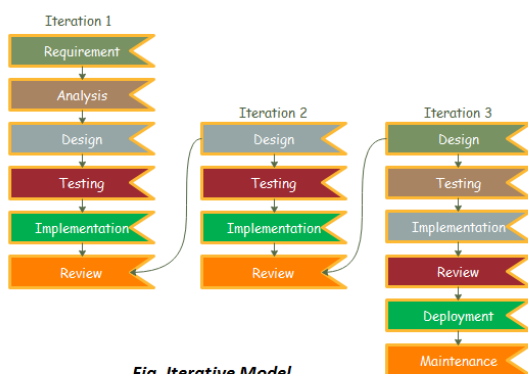


Fig. Iterative Model

When to use the iterative model ?

- When requirements are defined clearly and easy to understand

- When the software application is large
- When there is a requirement of changes in future.

Advantages (PROS) of iterative model :-

- Testing and debugging during smaller iteration is easy
- A parallel development can plan
- It is easily acceptable to ever changing needs of the project
- Risks are identified and resolved during iteration

Disadvantage (Cons) of iterative model

- ✓ It is not suitable for smaller projects
- ✓ More resources may be required
- ✓ Design can be changed again and again because of imperfect requirements
- ✓ Requirement changes can use over budget
- ✓ Project completion date not confirmed because of changing requirements.

Incremental Model

The incremental model in software engineering is a process model that breaks down the development process into small increments, or modules, that are developed and tested incrementally until the entire system is completed.

In the incremental model, each module is developed and tested separately, and then integrated into the larger system. This approach allows for early testing and detection of errors or issues and for adjustments to be made before the entire system is completed.

The incremental model is beneficial for large projects where it is difficult to predict all the requirements at the beginning of the development process. It allows for flexibility in adapting to

changing requirements or needs and for continuous feedback and improvement throughout the development process.

Like the waterfall model, the incremental model follows a linear sequential approach to software development, where the requirements are gathered, analyzed and defined at the beginning of the project. However, unlike the waterfall model, the development process is divided into smaller, more manageable modules or increments.

Each increment delivers a portion of the functionality required by the complete system, allowing developers to test and evaluate the software at each stage of the development process. Once an increment is completed, it is integrated with the previous increments to form a working system.

The difference between iterative and incremental models

In the iterative model, each iteration involves a complete cycle of requirement gathering, design, development, testing and deployment. The output of each iteration is a working product that can be evaluated by the stakeholders. The process then repeats with the next iteration, incorporating feedback and making improvements along the way.

In contrast, the incremental model involves dividing the project into smaller, self-contained pieces that can be developed and deployed independently. Each increment adds a new set of features or capabilities to the product, building on the work of previous increments.

The key difference between the two is that in the iterative model, each iteration involves a complete cycle of development, whereas in the incremental model, development is done in small increments that build on each other over time.

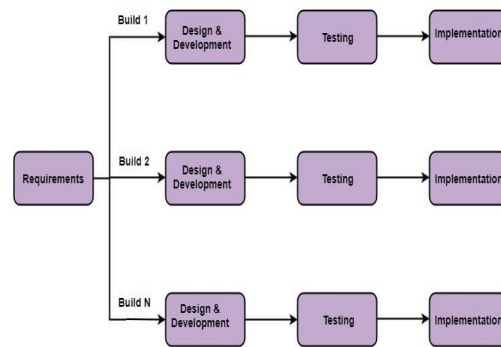


Fig: Incremental Model

Agile Model

The Agile SDLC (Software Development Life Cycle) is a methodology for developing software that emphasizes flexibility, speed, and collaboration. An agile is the time-confined, iterative approach to the software transmission that develops software from the initiating of the project to deliver it properly, up to the end.

Customer Collaboration: The customer, or end-user, is involved throughout the development process and provides continuous feedback on the software being developed. This helps to ensure that the software meets their needs and expectations.

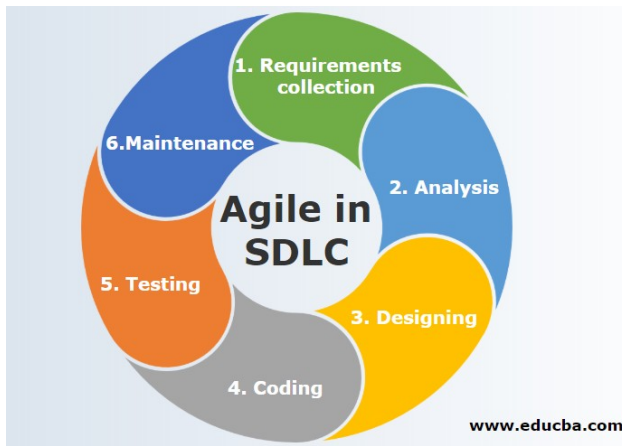
Emphasis on Individuals and Interactions: The Agile SDLC values people and communication over processes and tools. Team members work together closely, collaborate frequently, and communicate openly to ensure that everyone is on the same page.

Responding to Change: The Agile SDLC recognizes that requirements and priorities can change over the course of a project, and emphasizes the need to be able to adapt quickly and effectively.

Agile is a way of creating software that focuses on teamwork, communication, and flexibility. Instead of trying to create a big, complicated program all at once, Agile breaks it down into smaller parts called "user stories". Each user story represents a small piece of functionality that the software needs to have in order to be useful.

The team works together to decide which user stories are the most important, and they prioritize

them based on how essential they are to the software. Then they work on each user story in short periods of time called "iterations". During each iteration, the team works to complete a few user stories, and they test the software to make sure it's working correctly.



The team also communicates with each other and with the customer throughout the development process, to make sure everyone is on the same page and that the software is meeting the customer's needs. They make adjustments as they go, so that the software is always improving and adapting to changing requirements.

The Agile approach helps the team work more efficiently, because they're not trying to do too much at once. It also helps them catch problems early, because they're testing the software as they go, instead of waiting until the end. And because they're working closely with the customer throughout the development process, they can be sure that they're creating software that will actually be useful and valuable.

Let's simplify the agile approach

Imagine you want to build a big LEGO castle with your friends. The traditional way would be to plan everything in advance, like how many pieces you need, what the castle will look like, and then build it step by step following the plan.

But in the Agile model, we do things a little differently. Instead of planning everything at the beginning, we start with a small part of the castle, maybe just a tower. We build it quickly

and show it to everyone. Then we ask if they like it and if they want any changes. We listen to their ideas and make the changes right away.

Then we move on to the next part, like a gate or a wall. Again, we build it quickly and show it to everyone. We keep doing this, building small parts and getting feedback, making changes as we go along.

This way, we can see the castle taking shape gradually, and we can make improvements based on what we learn along the way. It's like building the castle piece by piece, getting feedback, and making it better as we go. It helps us work together as a team and make sure we're building something that everyone likes.

So, in the Agile model, we build software in small pieces, get feedback, and make improvements as we go along, just like building a LEGO castle bit by bit and making changes based on what we learn.

Agile is a way of working in software development that focuses on flexibility and collaboration. It's like being part of a sports team where everyone works together to score goals and win the game.

In the Agile model, instead of trying to plan everything from the beginning and following a fixed plan, we break the work into smaller parts called "sprints." Each sprint is like a mini-project with a specific goal. It's similar to playing a series of matches in a tournament.

During each sprint, the team works together to build a small piece of software, like a feature or a module. They have regular meetings called "stand-ups" where they discuss progress and any challenges they're facing. It's like gathering in the locker room to discuss tactics and strategies for the next match.

At the end of each sprint, the team reviews what they have built and gets feedback from customers or stakeholders. This feedback is important because it helps the team understand if they're heading in the right direction, just like analyzing

the game after each match to see what worked and what needs improvement.

Based on the feedback, the team adjusts their plans and decides what to work on in the next sprint. They can change their strategies or adapt to new information. It's like adjusting their game plan for the next match based on what they learned from the previous ones.

This cycle of sprints, feedback, and adjustments keeps repeating throughout the software development process. It allows the team to be flexible and respond quickly to changes or new requirements, just like a sports team that adjusts its tactics during the tournament.

Agile emphasizes teamwork, communication, and continuous improvement. It's like playing a sport where everyone has a role to play, and together, they strive to deliver the best software possible.

So, in simple terms, Agile in software engineering is about breaking work into smaller parts, working together as a team, getting feedback regularly, and adjusting plans based on what is learned along the way. It's like being part of a sports team where collaboration and adaptability are key to success.

Spiral Model

The Spiral model is a way of developing software that focuses on learning and improving as you go. It's like taking small steps forward while constantly checking and adjusting your path.

Imagine you have a big project, like writing a story. Instead of writing the whole story at once, you break it into smaller chapters. Each chapter is like a cycle in the Spiral model.

In each cycle, you plan and write a part of the story. Then, you pause to review what you've written and get feedback from others. This feedback helps you make the story better and catch any mistakes or things that can be improved.

After getting feedback, you move on to the next cycle, where you plan and write the next part of

the story. Again, you pause, review, and get feedback to make further improvements. You keep repeating this process until the story is complete.

The Spiral model is like climbing a spiral staircase. With each step, you make progress and get closer to your goal. But at each step, you also check your surroundings, see if there's a better path, and adjust your course if needed. It's a continuous process of learning, improving, and moving forward.

So, in simple terms, the Spiral model is like writing a story chapter by chapter. You write a part, get feedback, make improvements, and then move on to the next part. It's a step-by-step approach that allows you to learn and adjust along the way, making the story better with each cycle.

The Spiral model is a software development methodology that combines the iterative and incremental approach of the Agile model with the structured and systematic approach of the Waterfall model. It's called the Spiral model because it involves a series of cycles, or "spirals", each of which represents a phase in the development process.

Each spiral cycle involves four main steps: planning, risk-analysis, engineering, and evaluation. The planning phase involves defining the project objectives and requirements, and creating a plan for how the software will be developed. The risk analysis phase involves identifying potential risks and uncertainties that could affect the project, and creating strategies for dealing with them.

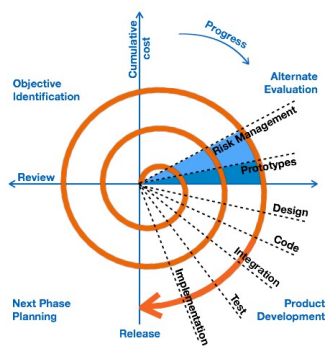
The engineering phase involves developing the software, using the plan and strategies created in the earlier phases. The evaluation phase involves testing the software to make sure it works correctly and meets the requirements.

The Spiral model is unique because it involves a continuous process of planning, risk analysis, engineering, and evaluation, with each spiral building upon the previous one. This allows for the software to be refined and improved over

time, and for potential risks to be identified and addressed early on.

Compared to other software development methodologies, the Spiral model is more flexible and adaptable, because it allows for changes and adjustments to be made throughout the development process. It also places a greater emphasis on risk management, which can help to prevent problems and delays later on.

Overall, the Spiral model is a flexible and adaptable approach to software development that emphasizes risk management and continuous improvement. It's a good choice for complex and high-risk projects, where it's important to have a structured and systematic approach while also allowing for flexibility and adaptation.



The Spiral model takes the best parts of both of these models and combines them into one. This model is called the Spiral model because it involves a continuous cycle of planning, development, and evaluation, with each cycle building upon the previous one in a spiral shape.

During each cycle, the team works on a small part of the software, testing and refining it as they go. They also analyze potential risks and problems that could occur during the development process, and create plans for how to deal with them.

The Spiral model is very focused on risk analysis, which means the team looks for potential issues or problems that could occur during the development process, and creates plans for how to deal with them. This helps to minimize the

impact of any issues that do arise, and keeps the development process on track.

The Spiral model also allows for incremental releases of the software, which means that the team can release parts of the software as they're completed, instead of waiting until the entire project is finished. This helps to keep the customer informed of the progress of the project, and allows them to provide feedback and make adjustments along the way.

Let's Define them in simpler terms

Imagine you want to bake a delicious cake, but you're not sure how it should turn out. The spiral model is like trying different recipes and making the cake better each time.

First, you start by mixing some basic ingredients like flour, sugar, and eggs. This is the first step of the spiral. You bake a simple cake and taste it to see if it's good. If it's not perfect, that's okay because it's just the beginning.

In the next step, you add more ingredients to make the cake better. Maybe you add some chocolate chips or vanilla extract to give it more flavor. You bake the cake again and see if it's better than before. If it's not quite right, you can still make changes and try again.

With each step, you add new things and improve the cake. You might try different frosting or decorations, like colorful sprinkles or fruit on top. Each time you bake the cake, you learn what works and what doesn't. You make adjustments to the recipe, like changing the amount of sugar or the baking time, based on what you learn.

The spiral keeps going, with each round making the cake better and better. You keep trying new things, making changes, and improving the cake until it becomes just the way you want it.

So, in the spiral model, software development is like baking a cake. You start with a basic version and keep adding new features and making improvements with each iteration, just like adding ingredients and making adjustments to the cake until it tastes amazing. It's all about learning

from each step and making the final software as good as it can be.

Another analogues definition

Imagine you have a big project to complete, like building a tree house. The Spiral model is like a step-by-step process where you keep improving and adding new things to make the tree house better each time.

First, you start by planning and designing the tree house. You think about how it should look, what materials you need, and what features you want to include, like a slide or a rope ladder. This is the first step of the spiral.

Next, you build a small part of the tree house based on your design. It could be the foundation or the base structure. Once you finish that part, you test it to see if it's strong and stable. If it's not quite right, you make adjustments and improvements.

Then, you move on to the next part, like adding walls or a roof. Again, you build it, test it, and make improvements if needed. Each time you add a new part, you check to make sure everything is working well together.

As you keep building and testing, you learn from your previous work and make the necessary changes. You might realize that you want to add more windows or make the tree house bigger. With each step, you keep improving and refining the design based on what you learn.

The spiral keeps going, with each round making the tree house better and better. You add new features, fix any problems, and make improvements based on your testing and feedback.

So, the Spiral model in software engineering is like building a tree house. You plan, build, test, and improve in a continuous cycle. Each time you go around the spiral, you add new parts and make adjustments based on what you learn. It's all about continuously improving the software,

just like adding new features and making the tree house better each time you work on it.

V-Model

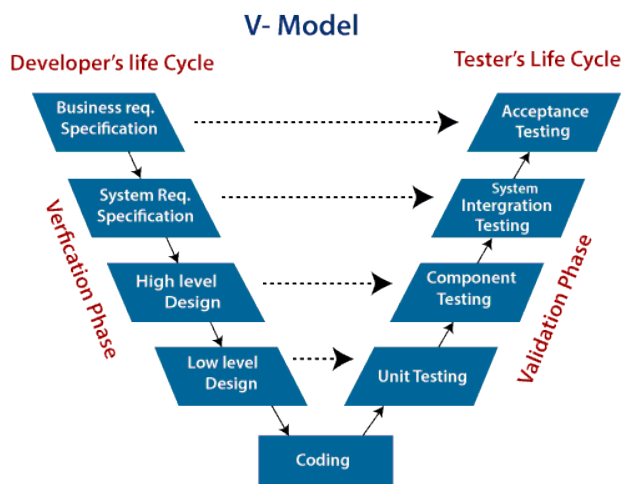
The V-Model is a software development methodology that is similar to the Waterfall model. It involves a sequential, step-by-step approach to software development, with each step building upon the previous one. However, the V-Model also emphasizes the importance of testing throughout the development process.

The V-Model is called the V-Model because the development process is shaped like the letter V. At the top of the V are the requirements gathering and analysis stages, which involve defining the project requirements and creating a plan for how the software will be developed. Then, the development process moves down the left side of the V, with each step involving more detailed design and development work.

At the bottom of the V is the testing phase, which involves testing the software to make sure it works correctly and meets the requirements. Then, the development process moves up the right side of the V, with each step involving more detailed testing and evaluation of the software.

The V-Model is unique because it places a strong emphasis on testing throughout the development process. This means that the software is tested at every step of the way, to ensure that it works correctly and meets the requirements. This helps to catch any issues or problems early on, before they become bigger and more expensive to fix.

Compared to other software development methodologies, the V-Model is more structured and formal, with a clear plan and set of steps that the team must follow. It's a good choice for projects where the requirements are well-defined and unlikely to change, and where there is a strong emphasis on testing and quality control.



Define it in simpler terms

Imagine you want to make a drawing of a cute animal. The V-model is like following a specific order to create your artwork step by step.

First, you start by drawing the outline of the animal. This is the top part of the "V." It's like drawing the basic shape of the animal, such as its body and head.

Next, you move down to the middle part of the "V." Here, you add more details to your drawing. You draw the eyes, nose, mouth, and other features that make the animal look cute and recognizable.

Then, you go to the bottom part of the "V." This is where you color your drawing. You use different colors to make the animal look vibrant and lively.

Once you finish coloring, you move up the other side of the "V." Here, you check your drawing for any mistakes or things that need improvement. If you find something that doesn't look right, you can make changes and fix it.

Finally, at the very top of the "V," you have your completed, beautiful drawing of a cute animal!

So, in simpler terms, the V-model is like following a specific order to create a drawing step by step. You start with the outline, add details, color, check for mistakes, and then you have your finished artwork. It helps you create

something awesome while making sure everything is just right along the way.

SOFTWARE DEVELOPMENT LIFE CYCLE COMPARISON

There are different ways to develop software, and each model has its own approach. Here are a few models explained in simpler terms:

1. **Waterfall Model:** The Waterfall model is like following a strict and straight path. You complete one phase before moving on to the next. However, this model doesn't work well when you need to make frequent changes because it doesn't allow for flexibility.
2. **Spiral Model:** The Spiral model combines elements of the Waterfall and Iterative models. It focuses on managing risks throughout the development process. While it offers some flexibility, it can be complex and time-consuming, making it less suitable for some projects.
3. **V-Model:** The V-Model is an extension of the Waterfall model. It emphasizes validation and verification at each stage. However, like the Waterfall model, it lacks flexibility and can be challenging to adapt to changes.
4. **Iterative Model:** The Iterative model involves developing the system in small steps, allowing for flexibility and adaptability. It's like taking small bites of a big sandwich. However, it may not provide the same level of continuous collaboration and improvement found in Agile methodologies.
5. **Incremental Model:** The Incremental model breaks the project into smaller parts, with each part adding new features to the system. It offers more flexibility than the Waterfall model, but it still lacks the continuous improvement and collaboration aspects found in Agile.

So, in simpler terms, each software development model has its own way of doing things. Some are rigid and sequential, while others are more flexible. Agile, which we haven't discussed yet, is another model known for its adaptability, collaboration, and continuous improvement.

UNIFIED MODELING LANGUAGE

UML (Unified Modeling Language) is a standardized modeling language used in software engineering to design and visualize software systems. It provides a graphical notation to represent various aspects of a software system, including its structure, behavior, and interactions between different components.

UML was developed by the Object Management Group (OMG) as a means of standardizing the various notations used on object-oriented software development. It is now widely used by software engineers, analysts, and developers to describe, design, and communicate the architecture of complex software systems.

UML provides various types of diagrams including use case diagrams, class diagrams, sequence diagrams, state diagrams, and activity diagrams to represent different aspects of the software system. These diagrams help developers and stakeholders to understand the system's architecture and behavior, making it easier to identify potential issues and communicate requirements and design decisions.

UML DIAGRAMS

- ➔ Class diagrams
- ➔ Use case diagrams
- ➔ Sequence diagrams
- ➔ Activity diagram
- ➔ State Machine Diagram
- ➔ Package Diagram
- ➔ Component Diagram

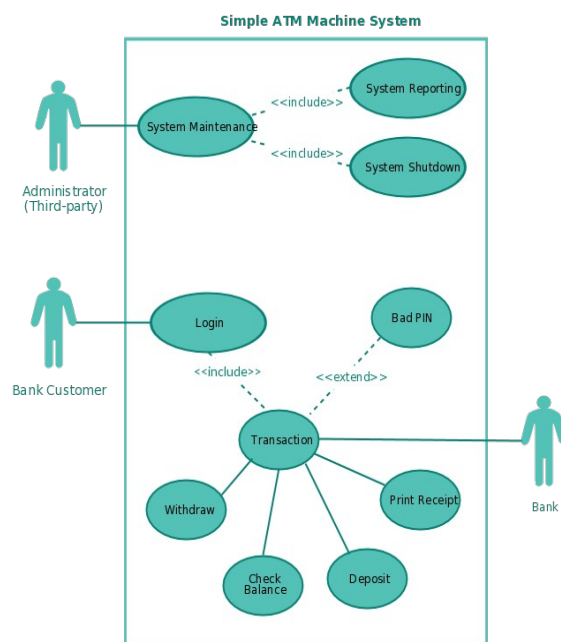
➔ Deployment Diagram

USE CASE DIAGRAM

In software engineering, a use case is a description of the intended functionality of a software system from the perspective of a user or a stakeholder. A use case describes a sequence of interactions between the user and the system that produces a result of value to one of the stakeholders.

Use cases can be represented visually using a use case diagram, which is a type of UML diagram. A use case diagram is composed of actors, use cases, and the relationships between them. An actor is a user or an external system that interacts with the system, while a use case represents a specific functionality of the system. The relationships between actors and use cases are depicted using lines.

How to draw a use case diagram



A use case is a representation of the functionality of a system, to create a use case diagram, it is important to follow certain rules to ensure that the diagram accurately represents the system's functionality in a clear and concise way.

Firstly, each actor or use case should be given an appropriate and meaningful name. This helps

to clearly identify and differentiate between the different components of the system.

Secondly , the communication between the actors and the use case must be defined in understandable way. This helps to ensure that the diagram accurately represents the interactions between the different components of the system.

Thirdly , specified notations should be used as and when required. This helps to ensure that the diagram follows a standard conventions and ways for others to understand.

Finally , the use cases should be enclosed with in a rectangle called the system boundary box , which indicates the scope of the system. All the use cases inside the rectangle represent the functionality that we intend to implement and anything outside the rectangle is considered out of scope for the system.

By following these rules , the use case diagram can be created in a structured and effective way , ensuring that it accurately represents the system's functionality and is easy to understand for all stake holders.

<< Include >> and << Extend >>

In UML use case diagrams , include and extend are two relationships between use cases that describe how they are related to each other.

Include :- An include relationship is used when one use case includes the functionality of another use case. It represents a common behavior that is reused in several use cases. The including use case adds additional behavior to the included use case. The included use case is mandatory and not optional.

In the include relationship , one use case (called the “base” use case) includes functionality of another use case (called the “included” use case). This means that the steps of the included use case are inserted in to the base use case wherever the include relationship appears.

For example , suppose we have two use cases , “create account” and “login”. The “Login” use case includes the “create account” use case , meaning that when a user attempts to log in but

does no have an account , this steps for creating an account will be included in the login process.

The include relationship is represented by a dashed arrow pointing from the base use case to the included use case , with the label «include» on the arrow. The base use case should reference the included use case in its description and the included use case should not reference the base use case.

Extend :- The extend relationship in use case diagrams is used to indicate optional behavior that can be added to a base use case. The optional behavior is represented by an “extension point” in the base use case. The extend use case is optional and not mandatory.

For example , consider a use case “order Product” which involves selecting a product and addition it to the cart , and checking out. However , the system may have an additional feature where the user can apply a discount code during check out. This is additional behavior can be represented as an extension of the “Order Product” use case.

In this case , we would draw an “extend” relationship from the “Order product” use case called “Apply Discounted code”. The “extend” relationship is shown with a dashed arrow pointing from the base use case to the extending use case. The extension point in the base use case would be labeled “Apply Discount Code”.

During the execution of the base use case , the system checks whether any extending use cases are applicable based on the extension point. If an extension is applicable , it is executed in the addition to the base use case. If no extensions are applicable , the base use case is executed as normal.

CLASS DIAGRAMS

UML (Unified Modeling Language) class diagrams are a type of static structure diagram that depict the classes , interfaces and their relationships to each other in a software system.

In a class diagram , a rectangle represents a class , with the class name appearing at the top. The class attributes and methods are listed in separate compartments inside the rectangle. The attributes

represent the properties or data associated with the class , while the methods represent the behaviors or actions the class can perform.

Relationships between classes are depicted by lines connecting the classes. There are several types of relationships , including inheritance (represented by a solid line with an arrow head pointing to the super class) , association (represented by a line connecting two classes) , and aggregation or composition (represented by a diamond – shaped arrow head pointing to the containing class)

The UML class diagram is a useful tool for visualizing the structure of a software system , and can aid in the design , implementation , and maintenance of a software applications.

The class diagram is a graphical representation of a system that displays the classes , their attributes and operations, along with the relationships between them. It is a type of static structure diagram that represents the static structure of the system.

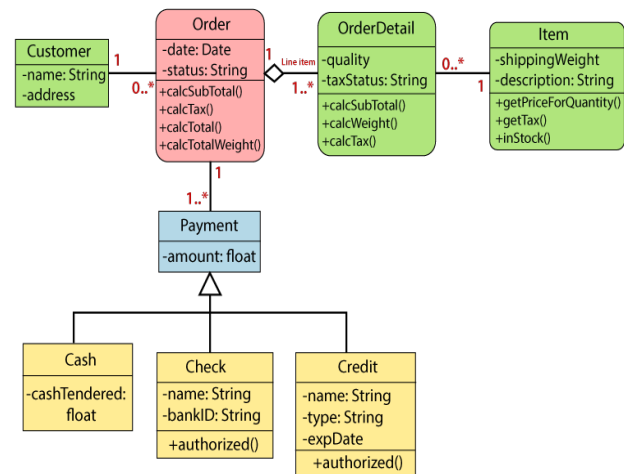
The upper section of the class diagram contains the name of the class , which represents the type of object that is being modeled. It is important to give meaningful name to the class so that it can be easily identified and understood. The name of the class should be capitalized and placed in the center of the upper section. It should be written in bold format to distinguish it from other elements in the diagram.

The middle section of the class diagram contains the attributes of the class , which describe the quality of the class. Attributes are the properties of an object that define its state. They are written along with their visibility factors , which indicate whether they are public (+) , private (-) , protected (#) or package (~). A meaningful name should be assigned to each attribute to explain it usage inside the class.

The lower section of the class diagram contains the methods or operations of the class. Methods define the behavior of an object and describe how it interacts with data. Each method is represented in the form of a list , where each method is written in a single line. The method's name should be descriptive , and clearly indicate

the behavior it performs. The parameters of the method , if any , should be listed.

In addition to these three sections , the class diagram also shows the relationships between classes. The relationships can be of different types such as association , aggregation and inheritance. These relationships help to define the behavior and structure of the system being modeled.

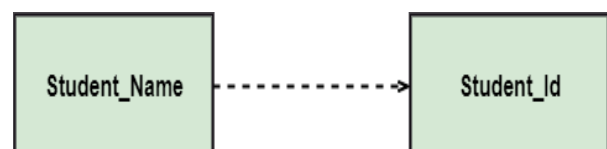


Relationships

Dependency :- In UML Dependency is a relationship between two elements , in which a change in one element affects the other element. It is represented as a dashed arrow with an arrow head pointing towards the element on which the other element depends.

Dependency relationship represents the concept of using or depending on another element in the system. It is a relationship in which one element , called the client , uses or depends on another element , called the supplier for its implementation or operation. The supplier provides service , function or resource that is needed by the client to operate.

For example , a car engine is dependent on the fuel system , which supplies fuel to the engine. If the fuel system fails , the engine will not operate properly. In this case , the engine represents the client , and the fuel system represent the supplier.

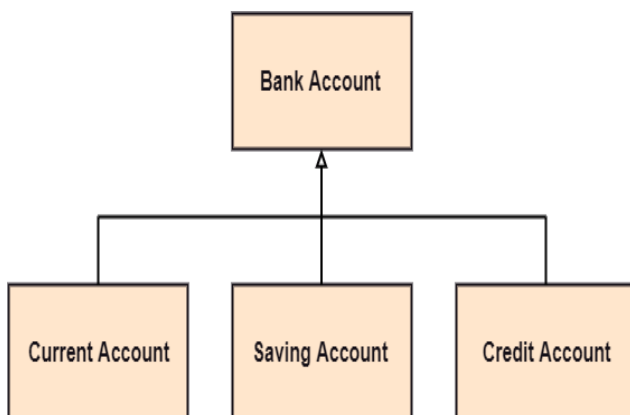


Generalization

In UML , generalization represents an inheritance relationship between two classes where one class is a more specific version of another class. The more specific class is called the sub class or the derived class and the more generalized class is called the super class or the base class.

Generalization is represented by an arrow with a hollow triangle at one end that points towards the base class , and the derived class is conducted to the base class with this arrow. The arrow indicates that the derived class inherits properties , attributes and behaviors from the base class.

Generalization allows the reuse of code and eliminates redundancy by promoting the creation of common attributes and behaviors in a super class that can be inherited by its sub class. It also facilitates the modeling of complex systems by allowing for the abstraction of common features in to a more general class.

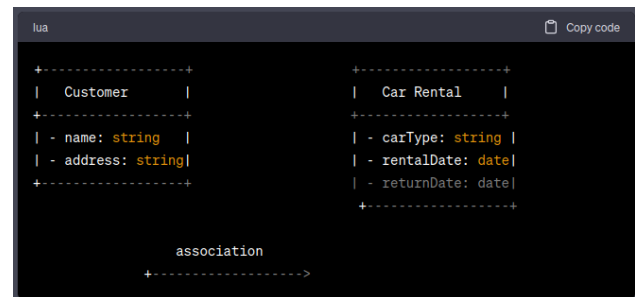


Association

In UML , association is a relationship between two or more classes that describes a connection or link between them. It shows that objects of one class are somehow related to objects of another class. Association can be represented using a solid line connecting the classes , with a possible arrow head indicating the direction of the relationship.

For example , consider a car rental system where a customer rents a car for a rental agency. In

this case , there is an association between the customer class and the car rental class , as a customer is associated with a specific car rental transaction. The association can be represented as follows :



The above example shows a uni-directional from customer to car rental. The arrow head pointing to the car rental class indicates that the association is navigable from the customer class to the car rental class but not vice versa.

Association can also have multiplicity , which defines the number of instances that can participate in the association. Multiplicity can be expressed using a range or a specific number. For example , a customer can rent multiple car rentals , while a car rental can only be associated with one customer at a time.

In UML , multiplicity is used to specify how many instances of one class are associated with (connected to) the instance of another class. It is represented as a range of numbers or specific values that define the number of occurrences of a class in a given association.

For example , a one to many relationship between two classes can be represented as "1..*" or "1 to many" in the multiplicity notation. This means that each instance of the first class is associated with one or more instances of the second class.

Similarly , a many to many relationship can be represented as "..." or "many to many" indicating that each instance of both classes can be associated with many instances of other class.

Multiplicity also includes minimum and maximum values to specify constraints on the number of associations, for example , a range of "0 ... 1" means that an instance of the class may or may not be associated with another instance of the class , while a range of "1..3" means that each

instance must be associated with at least one and up to three instances of another class.

AGGREGATION

In a class diagram, an aggregation relationship represents a connection between classes where one class is considered as a "whole" and another class is considered as a "part." It shows that the whole class contains or is composed of instances of the part class.

In simpler terms, an aggregation relationship in a class diagram shows that one class is made up of or contains instances of another class. It indicates a "part-of" or "has-a" relationship between the classes.

The aggregation relationship is represented by a line with a hollow diamond at the whole class's end, pointing towards the part class. The hollow diamond signifies the aggregation relationship. It shows that the part class is a component or part of the whole class.

For example, let's consider a class diagram for a car rental system. The system has two classes: "Car" and "RentalAgency." The "RentalAgency" class has an aggregation relationship with the "Car" class. This means that the "RentalAgency" contains or has instances of the "Car" class as part of its operations.

The aggregation relationship indicates that the "RentalAgency" class manages or has a collection of cars. Each car instance is considered a part of the rental agency's overall system.

In summary, an aggregation relationship in a class diagram represents the connection between classes where one class is considered a whole and another class is considered a part. It shows that the whole class contains or has instances of the part class. The

aggregation relationship is indicated by a line with a hollow diamond at the whole class's end.

Imagine you have a company with employees. Aggregation is like understanding how the company and its employees are connected.

In an aggregation relationship, the company is the "whole," and the employees are the "parts." It's like the company is made up of its employees, but the employees can still exist independently even if they are not part of the company anymore.

For example, let's say you have a company called "ABC Corp" and several employees working for it. The aggregation relationship shows that "ABC Corp" has a group of employees, but each employee can have their own existence and identity outside of the company. Even if one employee decides to leave or resign, the company "ABC Corp" will still continue to exist.

Think of it like a house and its rooms. The house is the "whole," and the rooms are the "parts." Each room can exist on its own, even if it's not part of that specific house anymore. The house can still stand and function even if a room is vacant or no longer part of it.

In an aggregation relationship, the parts are connected to the whole, but they maintain their independence. It's a way to represent that something is made up of smaller components, and those components can exist separately if needed.

So, in simpler terms, aggregation is like understanding how a company and its employees are connected. The company is made up of employees, but each employee can still exist independently. It's similar to how a house is made up of rooms, and each

room can stand on its own. Aggregation shows the relationship between the whole and its parts while recognizing the independence of the parts.

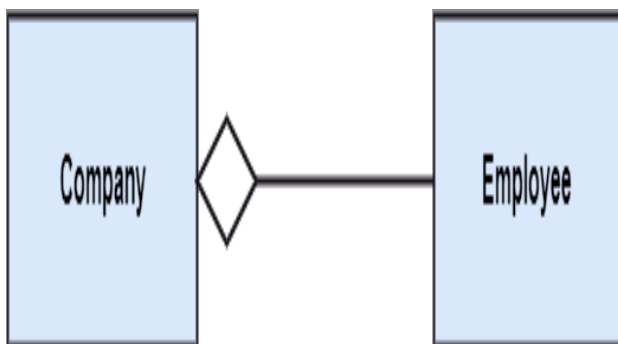
Imagine you have a collection of toys. Aggregation is like putting some toys together in a special box.

In a class diagram, aggregation shows that one class contains or holds another class as part of its collection. It's like having a big toy box that can hold different toys.

For example, let's say you have a "ToyBox" class and a "Toy" class. The "ToyBox" class can have an aggregation relationship with the "Toy" class. This means that the "ToyBox" class can hold or contain multiple instances of the "Toy" class.

Aggregation is shown in a class diagram using a line with a hollow diamond at one end pointing towards the class that contains the other class. It shows that the class with the diamond (like the "ToyBox" class) is the whole or container, and the class without the diamond (like the "Toy" class) is part of that whole.

So, in simpler terms, aggregation is like putting different toys together in a special box. The box (the whole) can contain many toys (the parts). It's a way to show that one class holds or contains instances of another class.



COMPOSITION

Imagine you have a puzzle set with different pieces. Composition is like when you create something using those puzzle pieces that cannot be taken apart easily.

In composition, there is a strong relationship between a "whole" and its "parts." If you remove the whole, the parts cannot exist on their own. They are tightly connected.

For example, let's think about a contact book and its contacts. The contact book is the whole, and the contacts are the parts. If you delete the contact book, all the contacts inside it will be lost because they only exist as part of that specific contact book.

Think of it as building a toy car using Lego blocks. The toy car is the whole, and the Lego blocks are the parts. If you take apart the toy car, the Lego blocks can be used for other things. But in composition, it's like using glue to permanently attach the Lego blocks to create the toy car. If you break the toy car, you cannot separate the blocks easily because they are now part of the car.

In simpler terms, composition is when you create something by putting parts together, and those parts cannot exist independently. They are dependent on the whole. If you remove the whole, the parts are lost too. It's like building something with puzzle pieces that are permanently connected, and if you break it, the pieces cannot be separated.

So, composition represents a whole-part relationship where the parts cannot exist without the whole, and if the whole is removed, the parts are lost.

Composition is like creating something by combining different parts, where the parts are essential to the existence and functioning of the whole. It represents a strong relationship between the whole and its parts.

Imagine you have a robot toy. The robot toy is made up of various components like a body, head, arms, and legs. In composition, these components are put together to form the complete robot toy.

The important thing about composition is that the parts are tightly connected and dependent on the whole. If you were to take apart the robot toy,

the individual parts wouldn't have much purpose or meaning on their own. They are designed to work together as a cohesive unit.

For example, let's consider a car and its wheels. The car is the whole, and the wheels are the parts. The wheels are an integral part of the car's structure and functionality. If you were to remove the wheels, the car would not be able to move properly or function as intended.

Similarly, think of a house and its rooms. The house is the whole, and the rooms are the parts. Each room is a crucial component of the house, serving different purposes such as a bedroom, kitchen, or living room. If you were to remove a room from the house, it would no longer be complete or functional.

In simpler terms, composition is like putting together different parts to create something whole. The parts are interconnected and rely on each other for the overall functionality and purpose of the whole. If you were to remove the whole, the parts wouldn't make much sense or serve their intended purpose individually. Composition represents a strong relationship where the parts are tightly connected to the whole.



How to draw a class diagram

A collection of class diagrams as whole represents a system. Some key points to keep in mind while drawing a class diagram are :-

1) To describe a complete aspect of the system , it is suggested to give meaningful name to the class diagrams.

2) The objects and their relationships should be acknowledged in advance.

3) The attributes and methods of each class must be known.

4) A minimum number of desired properties should be specified as more number of unwanted property will lead to a complex diagram.

5) Notes can be used as and when required by the developer to describe the aspects of the diagram.

6) The diagrams should be redrawn and reworked as many times to make it correct before producing its final version.

Usage of class diagrams

The class diagram is used to represent a static view of the system. It plays an essential role in the establishment of the component and deployment diagrams.

It helps to construct an executable code to perform forward and backward engineering for any system , or we can say it is mainly used for construction , it represents the mapping with object – oriented languages that are C++ , Java , etc.

Let's explore the different types of relationships in a class diagram:

1. One-to-One (1:1) Relationship: In a one-to-one relationship, one instance of a class is associated with exactly one instance of another class. It means that for every object in Class A, there is only one related object in Class B, and vice versa. This relationship is represented by a straight line between the two classes, with a "1" on one end and a "1" on the other end.

For example, consider the relationship between a Person and their Passport. Each person can have only one passport, and each passport is assigned to only one person.

2. One-to-Many (1:N) Relationship: In a one-to-many relationship, one instance of a class is associated with multiple instances of another class. It means that for every object in Class A, there can be multiple

related objects in Class B, but each object in Class B is related to only one object in Class A. This relationship is represented by a straight line with an arrow pointing from the "1" end to the "N" end.

For example, consider the relationship between a University and its Students. A university can have many students, but each student can be associated with only one university.

3. **Many-to-One (N:1) Relationship:** In a many-to-one relationship, multiple instances of a class are associated with one instance of another class. It means that for every object in Class A, there is exactly one related object in Class B, but multiple objects in Class A can be associated with the same object in Class B. This relationship is represented by a straight line with an arrow pointing from the "N" end to the "1" end.

For example, consider the relationship between a City and its Mayor. Many cities can have the same mayor, but each mayor is associated with only one city.

4. **Many-to-Many (N:N) Relationship:** In a many-to-many relationship, multiple instances of a class are associated with multiple instances of another class. It means that for every object in Class A, there can be multiple related objects in Class B, and vice versa. This relationship is represented by a line with crow's feet or dots on both ends.

For example, consider the relationship between Students and Courses. Many students can be enrolled in multiple courses, and each course can have multiple students enrolled.

These different types of relationships in a class diagram help depict how objects or instances of classes are connected and associated with each other. They provide a visual representation of the associations and dependencies between classes, helping to understand the structure and behavior of a system.

SEQUENCE DIAGRAMS

Sequence diagrams are a type of UML diagram used in software engineering to depict interactions between objects or components of a system. They are commonly used to visualize how different components of a system interact with each other over time.

In sequence diagram, time flows from top to bottom, with different objects or components depicted as vertical lines.

When we say that time flows from top to the bottom in a sequence diagram, it means that the diagram represents a chronological sequence of events in a system or process. Each vertical line in the diagram represents an object or component in the system and the horizontal arrows between the lines represent the messages or interactions between them. The diagram is read from top to bottom, with the events at the top occurring before those at the bottom. This allows us to understand the sequence of events and the order in which messages are sent and received between components. The sequence diagram is a powerful tool in software engineering for understanding complex systems and processes and designing effective software solutions.

Horizontal lines represent messages exchanged between these objects or components. These messages can include method calls, responses or exceptions.

The sequence diagram can also include various types of actors or stake holders, such as user or external systems. These actors are typically depicted as stick figures. They can be any person, system or organization that interacts with the system or software. These actors may trigger certain events in the system or respond to events generated by the system.

For example, in a banking system, a customer may be an actor who initiates the process of transferring funds. Similarly, a bank employee can be an actor who processes the customer's request. An external system, such as a payment gateway, can also be an actor who communicates with the banking system to complete the transaction.

The stick figures representing the actors are usually positioned outside the system boundary box to depict that they are external to the system. The sequence diagram then show how the messages or interactions flow between the actors and the system.

The interactions between actors and the system can be various types , such as sending a message , receiving a message , calling a method , or returning a value. These interactions are represented as arrows in the sequence diagram , with the arrows showing the direction of communication. The messages or interactions can be synchronous or asynchronous depending on whether they happen at the same time or not.

In the sequence diagram , actors or stakeholders are represented as stick figures to show their interaction with the system or software being designed. Actors

One of the benefits of sequence diagrams is that they allow developers to identify potential bottle necks or performance issues in the system. By visualizing the sequence of events and interactions , developers can identify areas where the system may be slow or inefficient.

The sequence diagrams are powerful tool for software engineers to visualize and design complex systems , as they allow for the clear representation of interactions and dependencies between components.

How to draw a sequence diagram

1. Identify the objects and actors involved in the scenario: The first step in drawing a sequence diagram is to identify the objects and actors involved in the scenario you are trying to model. Objects are the entities within the system that interact with each other, while actors are external entities that interact with the system.
2. Define the messages and interactions between the objects and actors: Once you have identified the objects and actors, you need to define the messages and interactions that occur between them.

This includes identifying the methods or functions that are called, as well as the data that is passed between them.

3. Determine the sequence of events: After defining the messages and interactions, you need to determine the sequence in which they occur. This involves identifying the order in which messages are sent and received, and the dependencies between different interactions.
4. Draw the diagram: Once you have identified the objects, actors, messages, interactions, and sequence of events, you can start drawing the sequence diagram. The diagram should include boxes representing the objects and actors, and arrows representing the messages and interactions between them. The sequence of events should be represented by the order of the arrows.
5. Add details and annotations: Finally, you can add details and annotations to the diagram to clarify the interactions and explain any important details. This may include labels for the objects and actors, descriptions of the messages and interactions, and notes on any important decisions or conditions that affect the sequence of events.

Here are some of the symbols used in a sequence diagram and their applications:

1. Objects: Objects are represented by rectangular boxes with the name of the object inside. Objects can represent anything from a person or a device to a software component or module.
2. Lifelines: Lifelines are vertical dashed lines that represent the lifespan of an object in the sequence diagram. They are drawn underneath the object box and indicate the period of time during which the object exists and can receive or send messages.

3. Messages: Messages are represented by arrows between objects or actors, and they indicate the communication between them. Messages can be synchronous (represented by a solid arrowhead) or asynchronous (represented by an open arrowhead). A message can also have a return value, which is represented by a dashed arrow.
4. Activation Bars: Activation bars are thin rectangles that show the time during which an object is executing a method or function. They are drawn on the lifeline of the object and indicate the period of time during which the object is busy processing a message.
5. Self-Message: A self-message is a message that an object sends to itself. It is represented by an arrow that points back to the same object.
6. Combined Fragment: A combined fragment is a group of messages that are related and can be shown together. They are represented by a box around the messages and can be used to show loops, conditions, and other types of control flow.

UML ACTIVITY DIAGRAM

Activity diagrams are a type of diagram in UML that are used to model the flow of control within a system. Unlike other UML diagrams, such as class diagrams or sequence diagrams, activity diagrams are not used to represent the implementation of a system, but rather the behavior of the system.

The primary purpose of an activity diagram is to illustrate the workflow of a system or process, showing how different activities are related to one another and how they are executed. Activities are represented by nodes in the diagram and the edges between the nodes represent the flow of control from one activity to another. The flow can be sequential, branched or concurrent, and to deal with these different

types of flows, activity diagrams include specialized symbols, such as forks and joins.

Activity diagrams are particularly useful for modeling complex systems or processes with multiple steps and decision points. By breaking down the workflow into individual activities and illustrating the relationships between them, activity diagrams can help developers and stakeholders better understand the behavior of the system and identify areas where improvements can be made.

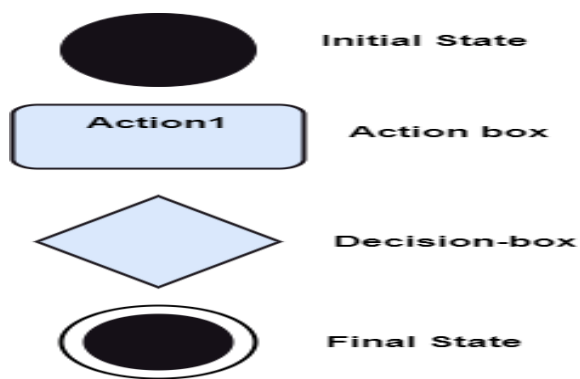
An activity diagram is a behavioral modeling diagram in unified modeling language (UML) to illustrate the flow control within a system. It shows the actions and states involved in a particular system, process or workflow. The activity diagram consists of various elements that represent different types of actions such as initial state, final state, decision box, and action box.

The initial state is represented as a circle with a black dot in the center, indicating the starting point of the set of actions. The final state, on the other hand, is represented by a circle with a concentric black border, which represents the end of the actions.

Decision boxes also known as diamond shapes, allow a control flow or object to follow one of the several paths based on certain conditions. This element helps in modeling decisions that occur in the workflow.

Action boxes, represented as rounded rectangles, show the set of actions that are to be performed. These actions can be sequential or concurrent, depending on the requirement of the system. An action box may contain multiple activities or tasks and it may have dependence on another action boxes.

Why we use activity diagram, briefly explain ?



An activity diagram is a modeling tool used to represent the behavior of a system in terms of **work flows and processes**. It is a type of event that encompasses a group of nodes and edges. Activity diagrams can be attached to various modeling elements such as use cases, interfaces, components and collaborations to model their dynamic behavior. They allow for the visualization and understanding of how various actions and tasks relate to each other and the order in which they occur. Though similar to flow charts, activity diagrams are not exactly the same, but they share the purpose of depicting the flow of activities between different nodes. They are useful for forward and reverse engineering of systems and processes.

How to draw an activity diagram

An activity diagram is a graphical representation of a business process or a system. It is a flowchart-like diagram that represents the workflow among various activities. The main purpose of an activity diagram is to represent the sequence of activities, actions, and workflows within a system. Activity diagrams are similar to flowcharts, but they are more powerful and flexible.

Activity diagrams are used in various domains, including software engineering, business process modeling, and project management. They help in identifying and analyzing the flow of activities within a system or process. This allows for better understanding of the system, identifying potential problems or bottlenecks, and improving the system or process.

Activity diagrams consist of several notations, including initial state, final state, decision box,

action box, swim lanes, branching, parallel flows, join nodes, control nodes, and forks. These notations allow for modeling of complex systems and processes, supporting exception handling, and enabling different types of flows and decision-making processes.

To create an activity diagram, the activities are first identified and analyzed to find out the constraints applied to them. Each and every activity, condition, and association must be recognized and modeled in the diagram. This helps in understanding the flow of activities and identifying any issues that need to be addressed.

Rules to be followed when drawing an activity diagram

When drawing an activity diagram, it is important to follow certain rules to ensure that the diagram is clear and understandable.

Firstly, each activity in the diagram should be given a meaningful name that accurately describes what the activity is doing. This helps to ensure that the diagram is easy to read and understand.

Secondly, all constraints that apply to the activities should be identified and included in the diagram. Constraints can include things like time limits, resource availability, or dependencies on other activities. By including all of the constraints, the activity diagram can be used to help plan and manage the activities more effectively.

Finally, all of the associations between the activities should be acknowledged in the diagram. This means showing how the activities are connected and how they relate to one another. This helps to provide a clear and complete picture of the workflow and ensures that all activities are properly accounted for.

When to use an Activity Diagram

An activity diagram can be used to portray business processes and workflows. Also, it is used for modeling business as well as the software. An activity diagram is utilized for the followings:

To graphically model the workflow in an easier and understandable way.

- To model the execution flow among several activities.
- To model comprehensive information of a function or an algorithm employed within the system.
- To model the business process and its workflow.
- To envision the dynamic aspect of a system.
- To generate the top-level flowcharts for representing the workflow of an application.
- To represent a high-level view of a distributed or an object-oriented system

Package Diagram

In this diagram , the different parts of the program are grouped in to “packages ” which are like boxes that hold related parts of the program. For example , there might be a package for all the parts of the program that deal with the user interface , and another package for all the parts of the program that deal with storing and retrieving data.

The packages are connected with arrows , which show how they depend on each other. For example , the user interface package might depend on the data storage package because it needs to access data from the storage package to display to the user.

Package diagram are useful because they help people understand how a program works and how the different parts of the program relate to each other. They can be used to design and test programs , or to communicate with other people about how a program works.

Deployment Diagram

A deployment diagram is like a map that shows how a computer program is installed and run on different devices or servers. It is a picture that helps people understand how the program is deployed and used in the real world.

In the diagram , the program is shown as a set of boxes and the devices or servers where it is installed and run are show as other boxes. For example , there might be a box for a desktop

computer , another box for mobile phone and another box for a server.

The boxes representing the devices or servers are connected by lines that show how they communicate with each other. For example , the desktop computer might connect to the server over the internet to send and retrieve data.

Deployment diagrams are useful because they help people understand how a program works in the real world , and how it interacts with different devices and servers. They can be used to plan and test the deployment of a program , or to communicate with other people about how a program is used.

COMPONENT DIAGRAMS

A component diagram is like a visual map that shows the different parts of a software or system and how they work together. Think of it as a picture that helps us understand how things are organized and connected.

In the diagram, each part is represented by a shape, like a puzzle piece or a building block. These parts are called "components." Each component has a specific job or function, like controlling a character in a game or handling the score.

To show how the components work together, lines or arrows are drawn between them. These lines represent the connections or relationships between the components. It's like drawing paths to show how one part interacts with another.

The component diagram also helps us understand how the components depend on each other. Some components need other components to work properly. For example, a scoring component might need information from a player component to keep track of points.

Overall, the component diagram helps us see the bigger picture of how all the different parts fit together and collaborate to make the software or system function correctly. It's like a map that guides us in understanding the structure and interactions of the different components.

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

In software engineering, requirements are statements that describe what a software system should do or how it should behave. They provide a foundation for designing and developing the software. Requirements can be categorized into two main types: functional requirements and non-functional requirements.

Functional Requirements: Functional requirements define the specific functionalities or features that the software system must possess. They describe the tasks, actions, or behaviors that the software should be able to perform. Functional requirements answer the question, "What should the software do?" Here are some key points about functional requirements:

1. **Describing specific actions:** Functional requirements outline the specific actions or operations that the software should be able to perform. For example, in a banking application, functional requirements may include actions such as "Allow customers to create new accounts" or "Enable users to transfer money between accounts."
2. **Identifying inputs and outputs:** Functional requirements specify the inputs that the software system should accept and the outputs it should produce. For example, a requirement may state that the software should take a customer's order as input and generate an invoice as output.
3. **Defining system behaviors:** Functional requirements describe the expected behavior of the software in response to different inputs. They outline how the software should respond or behave in various scenarios or use cases.

Non-Functional Requirements: Non-functional requirements specify the quality attributes or constraints that the software system should meet. They define how the system should perform or behave in terms of qualities like reliability,

performance, security, usability, and maintainability. Non-functional requirements answer the question, "How should the software perform?" Here are some key points about non-functional requirements :-

1. **Quality attributes:** Non-functional requirements focus on the qualities or attributes that the software should possess. These attributes include performance, reliability, security, usability, scalability, maintainability, and more.
2. **Constraints and limitations:** Non-functional requirements also include constraints or limitations that the software must adhere to. For example, constraints may involve compatibility with specific hardware or software platforms or compliance with legal or regulatory standards.
3. **User experience:** Non-functional requirements address aspects related to the user experience, such as usability, accessibility, and responsiveness. They ensure that the software is user-friendly and provides a satisfactory experience to its users.

In summary, functional requirements specify what the software system should do, describing the specific actions, inputs, and outputs. Non-functional requirements, on the other hand, define the qualities and constraints that the software should possess, addressing aspects such as performance, reliability, security, and user experience. Both types of requirements are crucial for designing and developing software systems that meet user needs and expectations.

CHAPTER FIVE

SYSTEM DESIGN

OVERVIEW A SYSTEM DESIGN

System design is the process of transforming the analysis model into a design model. In analysis, we focus on understanding what the user wants and needs. In design, we focus on figuring out how to build the system to meet those needs.

During system design, the developers set goals for the project and break down the system into smaller parts called subsystems. Each team can then work on building their assigned subsystem. Developers also make decisions on how to build the system, such as choosing the right hardware and software, managing the data that the system uses, controlling who can access the system, and planning how different parts of the system will work together.

The result of system design is a model that shows how the system is divided into subsystems, the goals that the design wants to achieve, and a clear description of the strategies chosen to build the system.

In simpler terms, system design is like creating a plan to build something. We figure out what the users want, and then we decide how to make it happen. We break the big project into smaller pieces, and different teams work on building their part. We also make choices about the tools and methods we'll use. The end result is a plan that shows how everything fits together and what we want to achieve.

SYSTEM DESIGN CONCEPTS

System design is a phase where we plan how to build a system based on what the users need. In this phase, we focus on breaking down the system into smaller parts called subsystems and figuring out how they will work together.

- 1) **Subsystem Decomposition** :- Subsystem decomposition means dividing the system into smaller parts called subsystems. We define each subsystem based on the

specific services it provides. It's like dividing a big task into smaller tasks to make it easier to handle.

1. **Subsystem Interface:** The subsystem interface is like the door through which the subsystem communicates with the rest of the system. We define the operations or functions that the subsystem provides, like buttons on a remote control.
2. **Properties of Subsystems:** Subsystems have two important properties: coupling and cohesion. Coupling refers to how much one subsystem depends on or is connected to another subsystem. Cohesion refers to how well the parts of a subsystem fit together and work towards a common goal.
3. **Techniques of Decomposition:** There are two common techniques to break down a system into subsystems: layering and partitioning. Layering is like building a cake with different layers, where each layer performs a specific function. Partitioning is like dividing a puzzle into pieces that fit together to form the complete picture.
4. **Architectural Style:** Architectural style refers to the overall design approach or pattern we use to build the system. It's like choosing a particular style for designing a house, such as modern, traditional, or futuristic. The architectural style guides how we structure and organize the subsystems.

In simpler terms, system design is like planning how to build a system based on what people need. We break the system into smaller parts called subsystems and define what each part does. We also decide how the parts will work together and how they communicate with each other. Subsystems have properties like how connected they are and how well their parts fit together. We use techniques like layering or partitioning to divide the system into subsystems. And just like choosing a style for a house, we

choose an architectural style to guide the overall design of the system.

SUBSYSTEMS

When designing a system, we break it down into smaller parts to make it easier to understand and manage. These smaller parts are called "subsystems." Subsystems are like separate pieces or modules within the system, and they have their own specific jobs to do.

To organize the application domain, we identify smaller parts called "classes" and group them into packages. This helps us manage the complexity of the system by organizing related classes together.

To manage the complexity of the solution domain, we decompose the system into subsystems. Each subsystem is made up of a collection of classes, associations, operations, events, and constraints that work closely together.

Subsystems have well-defined interfaces, which means they have clear ways of interacting with other parts of the system. They encapsulate the state and behavior of the classes they contain, which means they keep everything related to those classes together in one place.

In UML (Unified Modeling Language), subsystems are represented using component diagrams. These diagrams show how the subsystems fit together and interact with each other.

Each subsystem is designed to be manageable by a single developer or development team. This means that each subsystem represents a portion of the system that can be worked on independently.

For example, let's consider an accident management system. It can be decomposed into different subsystems, such as the DispatcherInterface subsystem that handles the user interface for the dispatcher, the FieldOfficerInterface subsystem that handles the user interface for field officers, the IncidentManagement subsystem responsible for creating and managing incidents, the ResourceManagement subsystem for tracking available resources like fire trucks and

ambulances, the MapManagement subsystem for displaying maps and locations, and the Notification subsystem for managing notifications.

In simpler terms, when designing a system, we break it into smaller parts called subsystems. Each subsystem has its own specific job and is made up of related classes. Subsystems have clear ways of interacting with other parts of the system and keep all the relevant information together. Subsystems are manageable by individual developers or teams. For example, an accident management system can be divided into different subsystems, each responsible for a specific part of the system, like the user interface, incident management, resource tracking, maps, and notifications.

In system design, we use a visual representation called UML (Unified Modeling Language) to show how the subsystems fit together. UML uses components to represent these subsystems.

Components in UML are shown as rectangles with a special icon in the upper right corner. They represent the different parts or modules of the system. Think of them as puzzle pieces that come together to form the complete system.

Dependencies among components, which show how they rely on each other, are depicted with dashed stick arrows. These arrows help us understand how the different parts of the system are connected.

In UML, components can represent both logical and physical components. A logical component is like a subsystem that doesn't have a physical form at runtime. It represents individual parts of the system's business logic that are combined together to form the complete system. It's like having different building blocks that make up the overall structure.

On the other hand, a physical component corresponds to a subsystem that has a physical form at runtime. It represents something like a database server or another tangible part of the system. It's like a specific piece of equipment that is essential for the system to work.

In simpler terms, when we use UML to show how different parts of the system fit together, we represent them as components. Components are like puzzle pieces that show the different subsystems or modules of the system. We use special shapes and icons to represent them. The connections or dependencies between the components are shown with dashed stick arrows. Components can represent either logical parts that are combined together or physical parts that have a specific form in the system, like a database server.

Properties of a subsystem : coupling and cohesion

Coupling and cohesion are important concepts when we design subsystems in a system.

Coupling refers to the level of dependency between two subsystems. If two subsystems are loosely coupled, they are more independent from each other. This means that making changes to one subsystem will have minimal impact on the other. On the other hand, if two subsystems are strongly coupled, changes in one subsystem can affect the other. It's better to have subsystems that are loosely coupled because it reduces the impact that errors or future changes in one subsystem have on the others.

For example, let's consider the IncidentManagement subsystem that interacts with a Database subsystem to store and retrieve incident records. In this case, there is high coupling between the Database subsystem and the three client subsystems (IncidentManagement, ResourceManagement, and MapManagement). If we make changes in the way data is stored in the database, we would need to make changes in all these client subsystems. To reduce this coupling, we can introduce a new subsystem called Storage. This new subsystem will act as a shield between the Database subsystem and the client subsystems. The client subsystems will use the services provided by the Storage subsystem, which will be responsible for issuing database queries. This reduces the direct dependency between the client subsystems and the Database subsystem.

Cohesion, on the other hand, refers to how closely related and focused the objects are within a subsystem. If a subsystem has high cohesion, it means that the objects inside it are closely related and perform similar tasks. If a subsystem has low cohesion, it means that the objects inside it are unrelated and perform different tasks. It's better to have subsystems with high cohesion because it means that the objects within the subsystem work well together towards a common goal.

When designing subsystems, there is a trade-off between cohesion and coupling. Breaking the system into smaller subsystems can increase cohesion, but it can also increase the number of interfaces and dependencies, leading to higher coupling. It's important to find a balance where we have subsystems with high cohesion and as low coupling as possible.

In simpler terms, coupling is like how much subsystems rely on each other. If they are loosely coupled, they are more independent, and changes in one subsystem don't affect the others much. Cohesion is about how well the objects within a subsystem work together. If they have high cohesion, it means they are closely related and focused on similar tasks. It's good to have subsystems that are loosely coupled and have high cohesion because it helps make the system more manageable and less prone to problems.

TECHNIQUES OF SUBSYSTEM : LAYERS AND PARTITIONS

When designing a system, we can organize it into different layers and partitions to manage its complexity.

Layers are like different levels or tiers of the system. Each layer groups together subsystems that provide related services. Think of layers as stacked on top of each other, with each layer depending on the services provided by the layer below it. Layers help organize the system and create a hierarchy.

A closed architecture means that a layer can only access the layer immediately below it. It can't

access higher layers. This keeps the dependencies between layers in a specific order.

An open architecture allows a layer to access layers at deeper levels, not just the immediate layer below it. This provides more flexibility in how the layers interact with each other.

Within a layer, we can further divide it horizontally into partitions. Partitions are like smaller groups of subsystems within a layer. They provide services to other partitions within the same layer. This helps in organizing and managing the subsystems within a layer.

Another approach to managing complexity is to divide the system into peer subsystems. Each subsystem is responsible for a different class of services. This allows different parts of the system to focus on specific tasks.

In general, when designing a system, we use a combination of layering and partitioning. We start by dividing the system into top-level subsystems, each responsible for specific functionality or running on a specific hardware node. If the system is complex, we continue breaking down the subsystems into lower-level layers and partitions until they become simple enough for a single developer to handle.

In simpler terms, when we design a system, we divide it into different layers, which are like different levels of the system. Each layer groups together related parts of the system. Layers have a specific order, and each layer depends on the services provided by the layer below it. We can also divide each layer horizontally into smaller groups called partitions. These partitions provide services to other partitions within the same layer. Another way to manage complexity is to divide the system into peer subsystems, where each subsystem handles a specific set of tasks. This helps make the system more organized and easier to work with.

Let's simplify it

Imagine you have a big puzzle to solve, but it has too many pieces and is very complicated. To make it easier, you can divide the puzzle into

different layers. Each layer groups together similar pieces that go together. You start with the bottom layer and add more layers on top. Each layer only uses the pieces from the layer below it.

Now, each layer can also have different parts called partitions. Think of these partitions as smaller groups of puzzle pieces within a layer. They work together and help each other within the same layer. This makes it easier to handle and organize the puzzle.

Sometimes, you can also divide the puzzle into different sections. Each section has its own special set of pieces to work on. This way, different parts of the puzzle can be solved by different people, and everyone can focus on their own tasks.

When we design a system, it's like solving a big puzzle. We use layers and partitions to make it easier to understand and manage. Each layer has its own pieces, and they depend on the pieces from the layer below. We can also divide each layer into smaller sections that work together. This helps us solve the puzzle step by step and make sure everything fits together nicely.

What are these layers and partitions

Layers and partitions are ways to organize and divide a system into manageable parts. Here are some examples of layers and partitions that can exist in a system:

Layers:

1. **Presentation Layer:** This layer deals with the user interface of the system, how it looks and how users interact with it.
2. **Business Logic Layer:** This layer contains the rules and logic that govern the operations and functionality of the system.
3. **Data Access Layer:** This layer is responsible for interacting with the database or data storage system to retrieve and manipulate data.
4. **Infrastructure Layer:** This layer handles the underlying technical aspects of the

system, such as networking, security, and system configuration.

Partitions within Layers:

1. User Management Partition: This partition handles user authentication, registration, and management.
2. Reporting Partition: This partition focuses on generating reports and analyzing data.
3. Payment Processing Partition: This partition handles payment transactions and integrates with payment gateways.
4. Inventory Management Partition: This partition deals with managing and tracking inventory levels and stock information.

These are just examples, and the actual layers and partitions in a system can vary depending on the specific requirements and architecture. The purpose of these divisions is to organize the system into logical components, making it easier to understand, develop, and maintain.

ARCHITECTURAL STYLES

When we are dealing with complex systems, it is important to plan how we divide the system into smaller parts. This division is called system decomposition. Once we start building the system, it becomes difficult to change the division later, as it would require many changes to the different parts of the system.

In system decomposition, we identify the different subsystems, which are like mini-systems within the bigger system. Each subsystem has specific tasks or services that it provides, and these subsystems are connected to each other in different ways. Sometimes, the connection is hierarchical, which means one subsystem is on top and controls the others underneath. Other times, the subsystems are equal and work together like a team.

An architectural style is like a pattern or template for how we organize the subsystems. It provides guidelines for how the subsystems should be structured and interact with each other. It helps us create a consistent design for

the system. Think of it like a blueprint or plan for building a house.

Software architecture is when we apply a specific architectural style to a real system. It's like actually building the house based on the blueprint. We implement the design and create the different subsystems according to the chosen architectural style.

There are many different types of architectural styles, and each one has its own characteristics and rules. Examples include client-server architecture, where one part of the system acts as a server providing services to clients, and layered architecture, where the system is divided into different layers with specific responsibilities. These architectural styles help us make the system organized, efficient, and easier to develop and maintain.

In simpler terms, when we have a big and complicated system, we need to plan how to break it down into smaller parts. We choose a design style, like a blueprint, that tells us how to divide the system and how the different parts should work together. Then, we actually build the system based on this design. There are different styles to choose from, like building a house in different architectural styles. Each style has its own rules and ways of organizing the system. This helps us make sure the system works well and is easier to manage.

TYPES OF ARCHITECTURAL STYLE

Repository Architectural Style: In this style, subsystems interact with a central data structure called the repository. Each subsystem can access and modify the repository, and they communicate with each other only through the repository. This style allows for the easy addition of new services by adding more subsystems. However, the main drawback is that the repository can become a bottleneck in terms of performance and making changes to the repository can affect all subsystems.

Model-View-Controller(MVC) Architectural Style: In MVC, subsystems are divided into three types: Model, View, and Controller. The Model

subsystem maintains the domain knowledge and implements the central data structure. The View subsystem is responsible for displaying information to the user, while the Controller subsystem manages the interactions with the user and notifies views of changes in the model. The model subsystems are designed to be independent of views and controllers.

Client/Server Architectural Style: In this style, subsystems are divided into clients and servers. Clients interact with the user and request services from servers. The request is usually done through remote procedure calls or object brokers. Control flow in clients and servers is independent, except for synchronization purposes. An example is an information system with a central database where clients handle user inputs and initiate database transactions, while servers perform the transactions and ensure data integrity.

Peer-to-Peer Architectural Style: In this style, subsystems can act as both clients and servers, meaning they can request and provide services to other subsystems. Each subsystem's control flow is independent, except for synchronization when making requests. Peer-to-peer systems are more complex than client/server systems and can introduce complications like deadlocks.

Three-Tier Architectural Style: This style organizes subsystems into three layers. The interface layer includes objects that interact with the user, such as windows or web pages. The application logic layer contains control and entity objects that handle the processing, rule checking, and notifications. The storage layer deals with storing, retrieving, and querying persistent data.

Four-Tier Architectural Style: This is an extension of the three-tier architecture with an additional layer. The interface layer is split into a Presentation Client layer (on user machines) and a Presentation Server layer (on servers). This allows for different presentation clients to be used while sharing some presentation objects across clients, reducing redundancy. For example, a banking system can have a web interface, an ATM interface, and an application client, with

shared forms processed in the Presentation Server layer.

These architectural styles help organize and structure subsystems within a system, making it easier to manage and maintain. Each style has its own characteristics and considerations, and the choice depends on the specific requirements and goals of the system.

CHAPTER 7

SOFTWARE QUALITY ASSURANCE

When we create software, we want to make sure it works correctly and doesn't have any problems. Software testing is like checking the software to find and fix any issues.

There are different types of software testing that we can use during the software development process. These tests help us make sure our software is stable and free from bugs.

Think of it like making a cake. When we bake a cake, we want to make sure it tastes good and doesn't have any problems. So, we do different things to check the cake and make it perfect.

Similarly, in software testing, we do different tests to check the software's functionality and make it bug-free. It's like checking each part of the cake to make sure it's cooked well and tastes delicious.

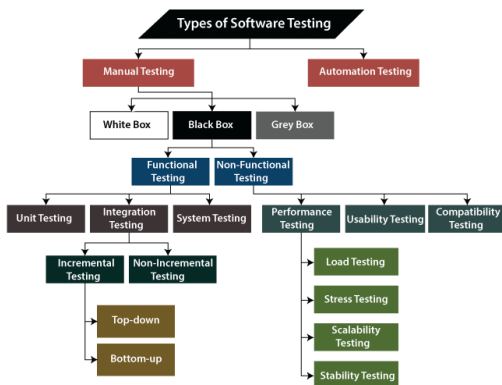
For example, one type of testing is called "functional testing." It's like tasting the cake to see if it has the right flavors and ingredients. Another type is called "performance testing." It's like checking if the cake is moist and fluffy.

There are many other types of testing, like "security testing" to make sure the software is safe, "user interface testing" to check if it's easy to use, and "compatibility testing" to see if it works well on different devices.

By performing these different types of testing, we can find and fix any problems in our software and make it stable and bug-free. It's like making

sure our cake is perfect and delicious before serving it to others.

So, in simpler terms, the various types of software testing are like different ways of checking our software to make sure it works correctly and doesn't have any issues. It's like making sure our cake tastes good and doesn't have any problems before we enjoy it.



MANUAL TESTING AND AUTOMATED TESTING

Software testing is a crucial process in software development that involves verifying and validating a software system to ensure that it meets the intended requirements and functions as expected. It involves executing tests, identifying defects or bugs, and providing feedback to improve the quality and reliability of the software.

Manual Testing: Manual testing refers to the process of executing tests and validating the software system manually, without the use of automated tools. In manual testing, a human tester performs various test cases, interacts with the software's user interface, and evaluates its behavior against expected results. Manual testing involves the following steps:

1. **Test Planning:** Testers analyze the software requirements, define test objectives, and develop test plans and test cases.
2. **Test Execution:** Testers execute the test cases, following specific steps and

verifying the system's behavior. They may input data, interact with the user interface, and manually validate the expected outputs.

3. **Defect Reporting:** If any defects or issues are discovered during testing, testers report them to the development team for resolution. They provide detailed information about the defect, including steps to reproduce it and any supporting documents or logs.
4. **Test Documentation:** Testers document the test results, including observed behaviors, issues, and any additional notes or feedback. This documentation helps in tracking the testing progress and providing a reference for future testing cycles.

Automation Testing: Automation testing involves using specialized tools and scripts to automate the execution of test cases, reducing manual effort and enhancing efficiency. Automation testing is suitable for repetitive tasks, regression testing, and large-scale projects. It involves the following steps:

1. **Test Script Development:** Testers write scripts or use testing frameworks to automate test cases. These scripts define the steps, interactions, and expected outcomes for each test.
2. **Test Execution:** The automated testing tool executes the test scripts, interacts with the software system, and compares the actual outcomes with the expected results.
3. **Defect Reporting:** If any defects are identified during automation testing, testers report them to the development team for resolution, similar to manual testing.
4. **Test Maintenance:** Test scripts may require periodic updates to accommodate changes in the software system. Testers maintain and update the scripts to ensure they

remain effective and aligned with the evolving software requirements.

Automation testing offers benefits like faster execution, repeatability, and wider coverage. However, it requires initial setup and continuous maintenance of test scripts.

Both manual testing and automation testing have their advantages and limitations. The choice between them depends on factors like project requirements, budget, complexity, and time constraints. Often, a combination of manual and automation testing is used to achieve comprehensive test coverage and maximize efficiency in software testing processes.

CLASSIFICATION OF MANUAL TESTING

Let's explain the classification of manual testing as black box, white box, and grey box testing in simpler terms.

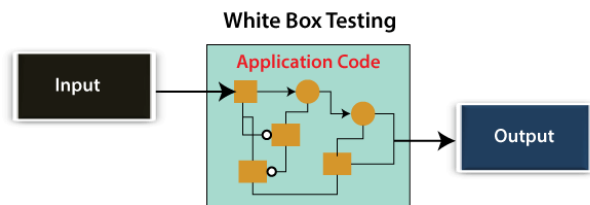


1. **Black Box Testing** :- Black box testing is like examining a closed box without knowing what's inside. In black box testing, the tester focuses solely on the software's external behavior and functionality, without knowing its internal structure or code.

Think of it as using a remote control for a TV. You don't need to know how the TV works inside; you just press buttons and see how it responds. Similarly, in black box testing, the tester interacts with the software's user interface and tests its inputs, outputs, and overall functionality without looking at the internal code.

2. **White Box Testing** :- White box testing is like having access to the inside of the box and examining its components. In white box testing, the tester has knowledge of the internal structure, logic, and code of the software being tested.

Imagine you are fixing a toy by opening it up and seeing how the gears and circuits work. In white box testing, the tester can review the software's internal code, algorithms, and system architecture. This allows them to create test cases specifically targeting those internal components and ensure they function correctly.



3. **Grey Box Testing** :- Grey box testing is like having a partially open box. In grey box testing, the tester has limited knowledge of the internal workings of the software but not the full understanding like in white box testing.

Think of it as peeking into the box and getting a glimpse of its internal structure without complete access. In grey box testing, the tester may have some knowledge of the internal code or database structure. This partial understanding allows them to design test cases that cover specific areas of interest or potential vulnerabilities.

In summary, the classification of manual testing into black box, white box, and grey box helps describe the level of knowledge and access a tester has to the internal workings of the software. Black box testing focuses on the software's external behavior, white box testing examines the internal structure and code, while grey box testing falls somewhere in between with limited knowledge of the internals.

TYPES OF BLACK BOX TESTING

Let's explain the types of black box testing, including functional and non-functional testing, in simpler terms.

Black box testing is like examining a closed box without knowing what's inside. It focuses on the

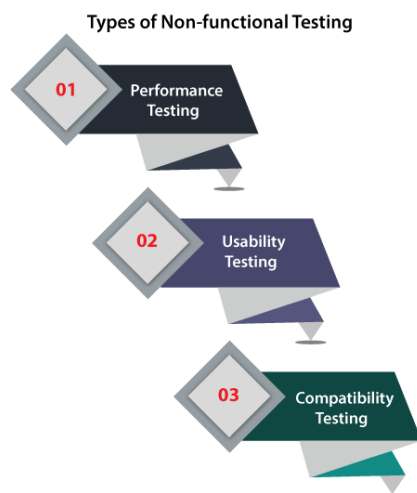
software's external behavior and functionality, without considering the internal code or structure.



1. **Functional Testing (Black Box):** Functional testing is like checking if the box does what it's supposed to do. In functional testing, the tester verifies that the software functions correctly according to the specified requirements or functional specifications.

Think of it as checking if a toy car moves forward and backward as expected. Similarly, in functional testing, the tester interacts with the software's user interface, provides inputs, and checks if it produces the expected outputs. It ensures that the software performs the intended functions correctly.

2. **Non-Functional Testing (Black Box):** Non-functional testing is like examining aspects of the box other than its primary function. It focuses on the software's qualities or characteristics beyond its functional behavior.



For example, consider checking if the toy car is durable, has good battery life, or is easy to use. Similarly, in non-functional testing, the tester evaluates aspects like performance, security, usability, reliability, and compatibility of the software.

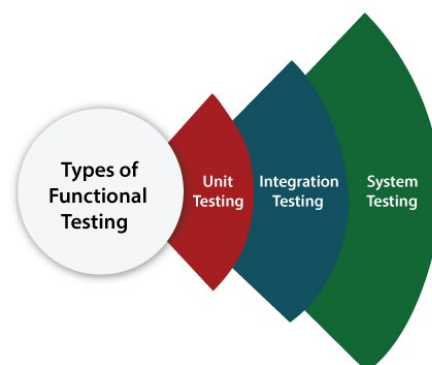
Here are a few examples of non-functional testing:

- **Performance Testing:** Checking if the software performs well under expected workloads and responds quickly.
- **Security Testing:** Evaluating the software's resistance to unauthorized access, data breaches, or other security vulnerabilities.
- **Usability Testing:** Assessing the software's user-friendliness and ease of use.
- **Reliability Testing:** Verifying the software's ability to function consistently and reliably over a period of time.
- **Compatibility Testing:** Ensuring that the software works properly across different devices, browsers, or operating systems.

In summary, black box testing involves examining the software's external behavior. Functional testing focuses on verifying if the software functions correctly as per the requirements. Non-functional testing evaluates the software's qualities beyond its basic functionality, such as performance, security, usability, reliability, and compatibility.

TYPES OF FUNCTIONAL TESTING

Let's explain the types of functional testing, including unit testing, integration testing, and system testing, in a brief detail:



1. **Unit Testing:** Unit testing is like checking each part of the box individually. In unit testing, individual components or units of the software are tested in isolation to ensure they work correctly on their own.

Think of it as testing each puzzle piece to make sure it fits and functions properly before putting the whole puzzle together. In unit testing, the focus is on testing small, independent units of code, such as functions or methods, to verify their expected behavior.

Unit testing helps identify any defects or issues within individual components early in the development process. It promotes modular and isolated testing of code segments and allows for easier debugging and maintenance.

2. **Integration Testing:** Integration testing is like checking if the different parts of the box fit and work together. In integration testing, the individual components or units tested in unit testing are combined and tested together as a group.

Imagine assembling the puzzle by connecting different puzzle pieces. In integration testing, the objective is to verify that the interactions and communication between the integrated components or units work as expected.

Integration testing ensures that the combined software components function correctly, exchange data properly, and maintain compatibility with each other. It helps identify any issues that may arise when multiple components interact, such as interface problems, data inconsistencies, or functional gaps.

3. **System Testing:** System testing is like checking the fully assembled box to ensure it works as a whole. In system testing, the complete software system is tested as a whole to validate its behavior against the specified requirements.

Think of it as checking the puzzle once it's fully assembled to ensure it forms the complete picture correctly. In system testing, the objective is to evaluate the overall functionality, performance,

and reliability of the software system from end to end.

System testing involves testing the system's interactions with external components, verifying the software's compliance with the specified requirements, and assessing its behavior in different scenarios. It aims to ensure that the software system meets the desired functionality and performs as expected in real-world usage.

In summary, the types of functional testing include unit testing, which checks individual components; integration testing, which verifies the interaction between components; and system testing, which evaluates the complete software system's behavior. These types of testing ensure that the software functions correctly at different levels of granularity, from individual units to the entire system.

CHAPTER FOUR

ANALYSIS

When we want to build something like a house or a toy, we first need to understand what it should look like and how it should work. The analysis phase in software engineering is just like that.

During the analysis phase, we talk to the people who will use the software and ask them lots of questions to understand what they want the software to do. It's like asking a friend what kind of toy they want and how they want it to play.

Once we understand what the software should do, we make a plan, or a model, of how it will work. It's like making a drawing or a blueprint of the toy we want to build. This plan helps us make sure we don't forget anything important and that the software will work correctly.

We also write down all the things the software needs to have, like buttons, colors, or sounds. It's like making a list of the features the toy should have, such as wheels, lights, or sounds.

In the analysis phase, we focus on understanding what the software needs to be and how it should behave. It's like figuring out what a toy needs to be and how it should play. We talk to people, make a plan, and write down all the important things.

By doing this analysis, we make sure that everyone understands what the software should do, and we can start building it in the best way possible. It's like making sure we know exactly what kind of toy to build before we start putting all the pieces together.

So, in simpler terms, the analysis phase in software engineering is like talking to people, making a plan, and writing down all the important things so that we understand what the software should do and how it should work. It helps us build the software in the best way possible, just like making sure we know what kind of toy to build before we start building it.

Analysis is a crucial phase in software engineering that focuses on understanding the requirements and needs of the users or stakeholders. It involves analyzing, documenting, and modeling the system to ensure that the software solution meets the desired objectives.

1. **Producing the Analysis Model:** The analysis phase aims to produce an analysis model, which is a representation of the system being developed. This model should be correct, complete, consistent, and verifiable. It serves as a blueprint that captures the essential aspects of the system, including its functionalities, behavior, and relationships.

The analysis model typically consists of various diagrams, documents, and artifacts that provide a comprehensive understanding of the system. These may include use case diagrams, activity diagrams, class diagrams, sequence diagrams, and more. The model helps stakeholders visualize and communicate the requirements and desired system behavior effectively.

2. **Structuring and Formalizing Requirements:** During analysis, developers work closely with stakeholders to elicit and understand the system requirements. This involves gathering information, conducting interviews, holding discussions, and analyzing existing documentation. The aim is to gather a clear understanding of what the system should do and how it should behave.

Once the requirements are gathered, the analysis phase involves structuring and formalizing them. Developers document the requirements in a systematic manner, ensuring they are clear, unambiguous, and comprehensive. This documentation serves as a reference for the subsequent phases of software development and helps in managing scope, expectations, and changes.

3. **Object-Oriented Analysis:** In object-oriented analysis, developers use the

principles and concepts of object-oriented programming to build a model that describes the application domain. They identify and define objects, their attributes, relationships, and behaviors. The focus is on understanding the real-world entities and their interactions, and then mapping them to software objects and classes.

The object-oriented analysis model helps in visualizing and representing the system in a way that aligns with the concepts of object-oriented programming. It allows for modular design, encapsulation, and re-usability, enhancing the overall efficiency and maintainability of the software solution.

In summary, the analysis phase in software engineering involves producing an analysis model that captures the system requirements, structuring and formalizing the gathered requirements, and, in object-oriented analysis, building a model describing the application domain. This phase sets the foundation for the subsequent phases of software development, ensuring a clear understanding of the system's objectives and requirements.

TYPES OF ANALYSIS MODEL

Let's define the three individual models in the analysis phase of software engineering:

1) Functional Model: The functional model helps us understand what the software should do and how it should behave. It is represented by use cases and scenarios.

- **Use cases:** Use cases are like stories that describe how people will interact with the software. They show different actions or tasks that users can perform and what the software should do in response. For example, in a shopping app, a use case could be "Adding items to the cart."
- **Scenarios:** Scenarios are like detailed examples of how the software should work in specific situations. They show step-by-step actions and expected results.

For example, a scenario for the shopping app use case could be "User selects an item, adds it to the cart, and sees the updated cart total."

The functional model helps the development team understand the software's functionality and how it should meet the users' needs.

2) Analysis Object Model: The analysis object model helps us understand the different objects or things that the software will use and how they are related. It is represented by class and object diagrams.

- **Class diagrams:** Class diagrams show different classes or types of objects in the software and how they are connected. It's like making a family tree, where each class represents a different type of object, and the connections show their relationships.
- **Object diagrams:** Object diagrams show specific instances of objects and how they interact. It's like taking a snapshot of a particular moment, showing the objects and their relationships at that point.

The analysis object model helps the development team understand the structure and relationships of objects in the software.

3) Dynamic Model: The dynamic model helps us understand how the software behaves over time and in different situations. It is represented by state machine and sequence diagrams.

- **State machine diagrams:** State machine diagrams show the different states or conditions that an object can be in and how it transitions between those states. It's like showing the different moods or emotions a person can have and how they change.
- **Sequence diagrams:** Sequence diagrams show the order of actions and interactions between objects in a particular scenario. It's like drawing a timeline that shows who does what and when.

The dynamic model helps the development team understand how objects change and interact with each other during the software's execution.

In summary, the analysis model consists of three individual models. The functional model shows what the software should do, the analysis object model shows the objects and their relationships, and the dynamic model shows how objects behave and interact over time. These models help the development team understand and design the software effectively.

In Simpler Terms

Functional Model: The functional model is like a map that shows us how the software should work. It's like a list of things the software needs to do to make it useful and fun for people to use.

For example, let's think of a video game. The functional model would describe all the different actions and tasks the game should have, like jumping, running, or collecting points. It helps the developers know what features to include in the game.

Analysis Object Model: The analysis object model is like a family photo album that shows different members of a family and how they are related. In this case, the "family" refers to the different objects or things the software will use.

In our video game example, the analysis object model would show all the different characters or objects in the game, like the player, enemies, or power-ups. It helps the developers understand what objects are needed and how they interact with each other.

Dynamic Model: The dynamic model is like a storyboard or comic strip that shows the different actions and events that happen in the software. It's like a series of pictures that tells a story.

In the video game example, the dynamic model would show how the player moves, how enemies appear, and how the game progresses. It helps the developers see the sequence of events and

understand how things change over time in the game.

So, in simpler terms, the analysis model is like a collection of different models that help the developers understand how the software should work. The functional model shows what the software needs to do, the analysis object model shows the different objects in the software, and the dynamic model shows how things happen over time. It's like having a map, a family photo album, and a storyboard to guide the developers in creating the software.

Let's simplify the explanation of working with the analysis object model and the dynamic model in software engineering:

When we work with the analysis object model or the dynamic model, it's important to remember that these models represent concepts that are easy for users to understand, not the actual technical details of the software.

For example, when we talk about the analysis object model, we focus on objects that make sense to users, like "User," "Product," or "Shopping Cart." We don't include technical concepts like "Database," "Subsystem," or "Network" because users don't need to know about them.

Similarly, when we talk about the dynamic model, we focus on how objects behave and interact in ways that users can relate to. We don't need to show the technical details like how data is stored in a database or how different components work together.

It's important to note that the analysis models are simplified versions of the actual software. The analysis object model may have classes that represent objects in the software, but they are more abstract and simplified compared to the actual software classes. The software classes have more details, like additional attributes and associations.

So, in simpler terms, when we work with the analysis object model and the dynamic model, we focus on concepts that users can easily

understand. We don't include technical details that users don't need to know. The analysis models provide a high-level overview, and later in the development process, the software will be implemented with more detailed classes and components.

ENTITY , BOUNDARY AND CONTROL OBJECTS

Let's simplify the explanation of the analysis object model and its different types of objects:

1. **Entity Objects** :- Entity objects are like the important information that the system keeps track of. They represent things that are stored or persisted in the system. For example, in the context of a watch, the entity objects could be things like the year, month, and day. These objects hold important data that the system needs to remember.

Entity objects are like important things that the system needs to remember. They are like puzzle pieces that hold important information. For example, in a watch, the entity objects could be the year, month, and day. These objects help the watch remember the current date

2. **Boundary Objects**: Boundary objects are like the bridges between the system and the people using it. They represent the interactions or connections between the users (actors) and the system. For example, in the watch example, the boundary objects could be things like buttons and the LCD display. These objects allow the users to interact with the watch and see the information displayed.

Boundary objects are like the parts that let us interact with the system. They are like buttons and screens that we can touch and press. For example, in a watch, the boundary objects could be the buttons we press to change settings or the screen that shows us the time.

3. **Control Objects**: Control objects are like the helpers that make things happen in the system. They represent the tasks that the user wants to perform and that the system supports. For example, in the watch example, the control object could be something like a "ChangeDateControl," which represents the activity of changing the date by pressing combinations of buttons. These objects are responsible for carrying out specific actions or functionalities in the system.

Control objects are like helpers that make things happen in the system. They are like the instructions or rules that tell the system what to do. For example, in a watch, there could be a control object called "ChangeDateControl" that helps us change the date by following specific steps. It's like having a special helper who tells the watch how to change the date.

To help distinguish between the different types of objects, we can use special labels called "stereotypes" in a modeling language called UML. For example, boundary objects may have the label "<<boundary>>" attached to their name, control objects may have the label "<<Control>>" attached to their name, and entity objects usually don't have any special label appended to their name, but sometimes they may have the label "<<entity>>."

In simpler terms, in the analysis object model, we have different types of objects. Entity objects hold important information, boundary objects connect the system with the users, and control objects represent the tasks that users want to do. For example, in a watch, the entity objects could be the date, month, and year; the boundary objects could be the buttons and the display; and the control object could be the one that handles changing the date when we press the buttons in a specific way. These different types of objects help us understand and represent the system in a clear and organized way.

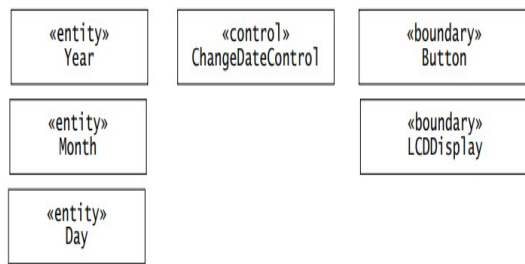


Figure 5-5 Analysis classes for the 2Bwatch example.

Artin

GENERALIZATION AND SPECIALIZATION

Let's simplify the definition of generalization and specialization in UML modeling:

1. **Generalization:** Generalization is like organizing things into groups or categories. It helps us see the big picture and find common traits among different things.

Think of it as grouping animals into categories like mammals, birds, or reptiles. We know that mammals have certain common characteristics like giving birth to live young and having hair or fur. Generalization helps us identify these common traits and group similar things together.

In UML modeling, generalization is when we identify a general concept or category at the top of a hierarchy, and we see that other concepts are more specific versions of that general concept. It's like saying that all mammals share common traits, but some mammals are more specific, like dogs or cats.

2. **Specialization:** Specialization is like zooming in and focusing on the details. It helps us identify specific traits or characteristics that set something apart from others in the same group.

Using the animal example, we can specialize within the mammal category to identify more specific animals like dogs, cats, or bears. Each of these animals has unique traits that differentiate them from other mammals. Specialization helps us understand these unique characteristics.

In UML modeling, specialization is when we identify more specific concepts or categories that inherit traits from a higher-level or more general concept. It's like saying that dogs have certain traits that are specific to them, but they also inherit traits from the general mammal category.

So, in simpler terms, generalization is like putting similar things into groups based on common traits, while specialization is focusing on the unique traits that set something apart within a group. In UML modeling, generalization helps us organize concepts into hierarchies, with a general concept at the top and more specialized concepts at the bottom. It helps us see common traits and relationships between concepts. Specialization, on the other hand, helps us identify more specific concepts within the hierarchy and understand their unique traits.

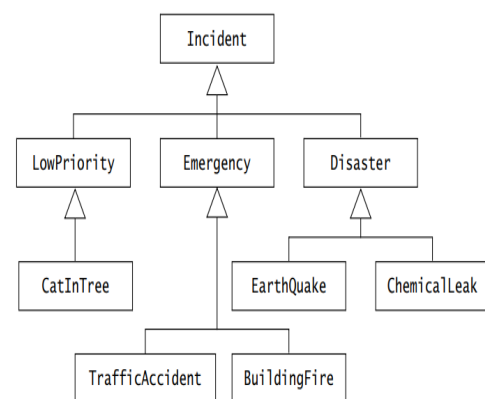


Figure 5-6 An example of a generalization hierarchy (UML class diagram). The top of the hierarchy represents the most general concept, whereas the bottom nodes represent the most specialized concepts.

ANALYSIS ACTIVITIES :- From Use cases to Objects

Let's simplify the explanation of the analysis activities in software engineering:

1. **Identifying Entity, Boundary, and Control Objects:** In this activity, we figure out the important things (entity objects) that the software needs to remember, the parts that let us interact with the software (boundary objects), and the helpers that make things happen in the software (control objects). It's like deciding what pieces are important, what buttons we

can press, and who helps us make things work.

2. **Mapping Use Cases to Objects with Sequence Diagrams:** In this activity, we connect the actions or tasks that users want to do (use cases) with the objects in the software. We create diagrams (sequence diagrams) that show the order of actions and interactions between objects. It's like drawing a storyboard to see how different objects work together to make the software do what users want.
3. **Modeling Interactions among Objects with CRC Cards and Identifying Associations:** In this activity, we create cards (CRC cards) for each object and write down their responsibilities and interactions. It's like making character cards for a story and describing what each character does. We also figure out how different objects are related or connected to each other (associations). It's like understanding how characters in a story interact or work together.
4. **Identifying Aggregates:** In this activity, we find groups of objects that work closely together or are strongly related. It's like recognizing a team of characters who always work together in a story. We call these groups aggregates. It helps us understand how different objects are organized and connected.
5. **Identifying Attributes:** In this activity, we identify the specific details or characteristics of objects (attributes) that are important for the software. It's like noting down the special features or qualities of the characters in a story.
6. **Modeling Inheritance Relationships:** In this activity, we identify the hierarchy or levels of objects where some objects are more general, and others are more specific. It's like understanding how different characters in a story are related, where some characters are more general

(like "animals") and others are more specific (like "dogs" or "cats").

7. **Reviewing the Analysis Model:** In this activity, we carefully check and review the analysis model we created. We make sure that everything makes sense and that we haven't missed any important details. It's like double-checking our work and making sure the story we've created is complete and accurate.

In simpler terms, the analysis activities involve figuring out important things, connecting user tasks with software objects, understanding how objects interact, recognizing groups of closely related objects, identifying specific details of objects, understanding the hierarchy of objects, and reviewing our work to ensure it's correct. It's like creating a story where we decide who the characters are, how they interact, and what special abilities or qualities they have, and then making sure everything fits together nicely.

IDENTIFYING ENTITY OBJECTS

When we want to build something like a toy or a game, we need to understand what it should do and how it should work. Requirements elicitation is like the first step in figuring out what our software should be like.

Let's simplify the explanation of the requirements elicitation process and the heuristics used to identify objects, attributes, and associations:

1. **Requirements Elicitation:** Requirements elicitation is like gathering all the important information we need to understand what the software should do. It involves looking at each task or action that users want to perform (use cases) and finding the important things (objects) that are involved in those tasks.
2. **Natural Language Analysis:** Natural language analysis is like carefully reading and understanding the words and sentences in a document that describes the software requirements. It helps us

identify the important parts and extract useful information from the text.

3. **Heuristics for Identifying Objects, Attributes, and Associations:** Heuristics are like helpful rules or guidelines that help us make decisions. In this case, we use heuristics to identify objects, attributes, and associations from the requirements document.

Here are some heuristics we can use:

- **Terms that need clarification:** We look for words or phrases that are not clear or need more explanation. It's like finding parts of the requirements that we need to ask questions about to understand better.
- **Recurring nouns:** We look for words that appear frequently in the requirements. It's like noticing names or things that are mentioned multiple times.
- **Real-world entities:** We look for names of things in the real world that the software needs to keep track of. It could be people, places, or things. It's like recognizing important characters or objects in a story.
- **Real-world activities:** We look for actions or tasks that the software needs to track. It's like identifying important actions or events in a story.
- **Data sources or sinks:** We look for things that the software needs to interact with, like getting data from a printer or sending data to another system. It's like recognizing objects that the software needs to connect with.

In simpler terms, during requirements elicitation, we carefully read and understand the information provided to identify important things and make sense of what the software should do. We use helpful rules like looking for recurring words, important entities, activities, and things that the software needs to interact with. It's like finding characters, objects, and actions in a story and

understanding how they fit together in the software.

EXAMPLE :- Let's simplify the explanation of the use case and its related entity objects:

Use Case: ReportEmergency This use case is about a task called "Report Emergency" that involves two people, the FieldOfficer (a police or fire officer on duty) and the Dispatcher (a police officer who manages incidents). The FieldOfficer initiates the task, and it requires communication with the Dispatcher.

Flow of Events:

1. The FieldOfficer starts the "Report Emergency" task on her device.
2. The software presents a special form to the FieldOfficer, like a digital paper with fields to fill in.
3. The FieldOfficer fills in the form by selecting the emergency level, type, location, and giving a brief description of the situation. She also describes possible responses to the emergency situation.
4. Once the FieldOfficer completes the form, she submits it.
5. The software receives the form and notifies the Dispatcher, like sending a message to get the Dispatcher's attention.
6. The Dispatcher reviews the information in the form and creates an "Incident" entry in the system's database using a special function called "OpenIncident." The Dispatcher selects a response to the reported emergency and acknowledges the report.
7. The software shows the acknowledgment and the selected response to the FieldOfficer, like displaying a message on her device.

Entity Objects: In this use case, there are specific things or concepts that play important roles. We call these things entity objects.

- **Dispatcher:** This is like a police officer who manages incidents. They receive reports and make decisions on how to respond to emergencies.

- **EmergencyReport:** This is like the initial report about an incident made by the FieldOfficer to inform the Dispatcher. It includes details about the emergency situation.
- **FieldOfficer:** This is like a police or fire officer who is on duty and reports emergencies to the Dispatcher.
- **Incident:** This is like a situation that requires attention from a FieldOfficer. It could be something like a crime or an accident.

In simpler terms, the use case "ReportEmergency" is a task where a FieldOfficer on duty reports an emergency to a Dispatcher. The FieldOfficer fills in a form with details about the emergency, and the software notifies the Dispatcher. The Dispatcher reviews the information, takes action, and acknowledges the report. During this process, there are important things involved, like the people (FieldOfficer and Dispatcher), the report itself (EmergencyReport), and the incident that needs attention.

IDENTIFYING BOUNDARY OBJECTS

Let's simplify the explanation of boundary objects and the heuristics used to identify them:

Boundary objects are like the parts of the software that help us interact with the people using it. They are like the bridges between the system and the actors (the people using the software).

In each task or action that people want to do (use case), they interact with at least one boundary object. The boundary object collects the information from the person and translates it into a form that the software can understand and use.

For example, think of a button on a screen or a form where we enter information. These are examples of boundary objects. They help us communicate with the software by pressing buttons or filling in information.

To help us identify boundary objects, we have some helpful rules called heuristics. These rules

guide us in recognizing what things are boundary objects.

Here are some heuristics we use:

- **User Interface Controls:** We look for buttons or controls that people need to use to start a task. For example, a "Report Emergency" button is a control that people can press to start reporting an emergency.
- **Data Entry Forms:** We look for places where people need to enter information into the software. For example, an "Emergency Report Form" is a place where people can fill in details about an emergency.
- **System Notices and Messages:** We look for messages or notices that the software shows to people in response to their actions. For example, an "Acknowledgment Notice" is a message that the software displays to let people know that their action has been received.
- **Actor Terminals:** When multiple people are involved in a task, we use special names for the parts of the interface that each person interacts with. For example, a "Dispatcher Station" is the part of the interface that the dispatcher uses.

It's important to always use words that the people using the software will understand. We should not use technical words or terms that are only used by the people building the software. We want to make sure everyone can understand how to interact with the software.

In simpler terms, boundary objects are like the parts of the software that help us interact with it. They are like buttons and forms that we use to communicate with the software. To find these boundary objects, we look for buttons, forms, messages, and special names for different people's parts of the interface. It's important to use words that everyone can understand.

EXAMPLE :- Let's simplify the explanation of the use case and its related boundary objects:

Use Case: ReportEmergency This is a task that involves two people, the FieldOfficer and the Dispatcher. The FieldOfficer initiates the "Report Emergency" function using her device, and it involves communication with the Dispatcher.

Flow of Events:

1. The FieldOfficer activates the "Report Emergency" function on her device.
2. The software presents a form to the FieldOfficer, like a special page with fields to fill in.
3. The FieldOfficer fills in the form by selecting the emergency level, type, location, and giving a brief description of the situation. Once done, she submits the form.
4. The software receives the form and notifies the Dispatcher.
5. The Dispatcher reviews the information in the form and creates an incident in the system's database using the "OpenIncident" function. The Dispatcher selects a response and acknowledges the report.
6. The software displays the acknowledgment and the selected response to the FieldOfficer.

Boundary Objects: In this use case, there are certain parts of the software that help in the interaction between the FieldOfficer and the system. These parts are called boundary objects.

- ReportEmergencyButton: This is like a special button that the FieldOfficer presses to start the "Report Emergency" task.
- AcknowledgmentNotice: This is like a notice or message that shows the Dispatcher's acknowledgment to the FieldOfficer.
- EmergencyReportForm: This is like a special form that appears on the FieldOfficer's device when she selects the "Report Emergency" function. It has fields where she can enter information about the emergency.

- DispatcherStation: This is like a computer used by the Dispatcher to review and handle the reported emergencies.

In simpler terms, the use case "ReportEmergency" is a task involving a FieldOfficer and a Dispatcher. The FieldOfficer starts by pressing a special button and filling in a form about an emergency. The software then notifies the Dispatcher, who reviews the information and takes appropriate action. During this process, there are special objects like buttons, notices, forms, and computers that help with the interaction between the people and the software.

IDENTIFYING CONTROL OBJECTS

Let's simplify the explanation of control objects and the heuristics used to identify them:

Control objects are like the coordinators or managers in a use case. They are responsible for making sure things happen in the right order and coordinating the actions of other objects. Control objects are usually created at the beginning of a use case and stop existing once the use case is finished.

To help us identify control objects, we have some helpful rules called heuristics. These rules guide us in recognizing what things are control objects.

Here are some heuristics we use :-

- **One Control Object per Use Case:** We have one control object that oversees the entire use case. It's like having one person in charge of making sure everything goes smoothly.
- **One Control Object per Actor:** If multiple people are involved in a use case, we have one control object for each person. Each control object is responsible for coordinating the actions of that particular person.
- **Lifespan of a Control Object:** The control object should exist throughout the whole use case or for as long as the person is interacting with the software. It's like the

control object is active as long as the person is using the software.

Now, let's see how these control objects work in the "ReportEmergency" use case:

- **ReportEmergencyControl:** This control object manages the reporting function for the FieldOfficer. It is created when the FieldOfficer selects the "Report Emergency" button. It presents a form (EmergencyReportForm) to the FieldOfficer and collects the information from the form. It creates an EmergencyReport and sends it to the Dispatcher. It waits for an acknowledgment from the Dispatcher and, upon receiving it, creates an AcknowledgmentNotice and shows it to the FieldOfficer.
- **ManageEmergencyControl:** This control object manages the reporting function for the Dispatcher. It is created when an EmergencyReport is received. It displays a form (IncidentForm) to the Dispatcher. Once the Dispatcher creates an Incident, allocates resources, and submits an acknowledgment, the ManageEmergencyControl forwards the acknowledgment to the FieldOfficer.

In simpler terms, control objects are like managers or coordinators in a use case. They make sure things happen in the right order and coordinate the actions of other objects. For the "ReportEmergency" use case, we have control objects for the FieldOfficer and the Dispatcher. The FieldOfficer's control object oversees the reporting process, while the Dispatcher's control object manages the handling of the report. They ensure that the right forms are displayed, information is collected, and acknowledgments are exchanged.

MAPPING USE CASES TO OBJECTS WITH SEQUENCE DIAGRAMS

Let's simplify the explanation of sequence diagrams and the heuristics used to draw them:

Sequence diagrams are like visual representations of how different objects interact with each other in a use case. They show the behavior and communication between objects over time.

In a sequence diagram:

- Columns represent the objects that are involved in the use case. The leftmost column is usually the person who starts the use case (the actor).
- Horizontal arrows represent messages or signals being sent from one object to another.
- When an object receives a message, it triggers an action called an operation.
- The activation of an operation is shown as a vertical rectangle that represents the time the operation is active.

Sequence diagrams also help us understand the lifetime of objects involved:

- A dashed line between an object's rectangle and the bottom of the diagram represents the time span when the object exists.
- Objects that already exist before the sequence diagram starts are shown at the top.
- Objects created during the interaction are represented by a "create" message pointing to the object.
- Objects that are no longer needed are shown with a cross, indicating when they cease to exist.

To draw sequence diagrams, we follow some helpful rules called heuristics:

- The first column should represent the actor who initiated the use case.
- The second column should be a boundary object, like a button or control, that the actor used to start the use case.

- The third column should be the control object that manages the rest of the use case.
- Control objects are created by boundary objects when they initiate use cases.
- Boundary objects are created by control objects.

In simpler terms, sequence diagrams are like pictures that show how objects talk to each other in a use case. The columns represent the objects involved, and the arrows show the messages they send. The length of the rectangles shows how long an action takes. We can also see when objects exist, when they are created, and when they are no longer needed. To draw these diagrams, we start with the actor, then show the boundary object they use, and finally the control object that manages everything else. Control objects create boundary objects, and boundary objects create control objects.

MODELING INTERACTIONS AMONG OBJECTS WITH CRC CARDS

Let's simplify the explanation of CRC (Class, Responsibilities, and Collaborators) cards and their relationship with sequence diagrams:

CRC cards are a way to understand and organize the interactions among objects in a software system. Each class is represented on an index card, called a CRC card. The card contains important information about the class, such as its name, responsibilities, and the names of other classes it collaborates with.

The CRC card has three sections :-

- The top section shows the name of the class.
- The left column lists the responsibilities of the class, which are the things it needs to do or take care of.
- The right column lists the names of other classes that the class works together with to accomplish its responsibilities.

CRC cards and sequence diagrams are two different ways to represent and understand

interactions among objects, but they serve the same purpose.

Sequence diagrams are like pictures that show how objects communicate and pass messages to each other over time. They are useful for a single person or modeler to understand and visualize the flow of actions in a use case or scenario.

On the other hand, CRC cards are physical index cards that can be used by a group of developers. Each developer can have their own set of CRC cards, and they can use them to discuss and collaborate on understanding the responsibilities and collaborations between classes. It helps the group have a shared understanding and organize their work.

In simpler terms, CRC cards are like special cards that describe classes in a software system. They have the class name, responsibilities (what the class does), and the names of other classes they work with. CRC cards are useful for a group of developers to understand and work together on a project. Sequence diagrams are like pictures that show how classes communicate, and they are useful for a single person to understand the flow of actions in a use case. Both CRC cards and sequence diagrams help us understand how different classes interact in a software system.

IDENTIFYING ASSOCIATIONS AND ATTRIBUTES

Let's simplify the explanation of associations and the heuristics used to identify them:

Associations show relationships between different classes in a software system. For example, one class called FieldOfficer is associated with another class called EmergencyReport because the FieldOfficer writes the report.

Identifying associations has two advantages :-

1. It makes the relationships between objects clear in the analysis model. For example, it explicitly shows that an EmergencyReport can be created by either a FieldOfficer or a Dispatcher.
2. It helps the developer discover special cases or exceptions related to the associations. These special cases are called

boundary cases and need to be clarified in the model. For example, should the system allow multiple FieldOfficers to write one EmergencyReport? Should it allow anonymous EmergencyReports? These questions should be discussed with clients or end users during the analysis phase.

Associations have several properties:

- **Name:** Each association has a name that describes the relationship between the classes. For example, the name "Writes" could be used to describe the association between FieldOfficer and EmergencyReport.
- **Role:** Each class in the association has a role that describes its function with respect to the association. For example, the FieldOfficer plays the role of the author in the "Writes" association.
- **Multiplicity:** Each end of the association has a multiplicity that describes the possible number of instances. For example, the "*" symbol indicates that a FieldOfficer can write zero or more EmergencyReports, while the "1" symbol indicates that each EmergencyReport has exactly one FieldOfficer as the author.

To identify associations, we can use Abbott's heuristics :-

- **Examine verb phrases:** Look for phrases like "has," "is part of," "manages," "reports to," "is triggered by," "is contained in," "talks to," or "includes" to identify associations.
- **Name associations and roles precisely:** Give associations and roles clear and descriptive names.
- **Use qualifiers:** Use qualifiers to identify specific namespaces or key attributes related to the associations.
- **Eliminate redundant associations:** Remove any association that can be derived or understood from other associations.
- **Don't worry about multiplicity at first:** Focus on identifying associations before

determining the number of instances involved.

- **Avoid too many associations:** Having too many associations can make the model hard to understand.

In simpler terms, associations show relationships between different classes in a software system. They help us understand how classes are connected and work together. Associations have names, roles, and multiplicity to describe the relationship and the number of instances involved. To identify associations, we look for specific words or phrases that describe the relationship. It's important to name associations and roles clearly, use qualifiers when needed, and avoid having too many associations in the model.

ATTRIBUTES

Let's simplify the explanation of attributes and the heuristics used to identify them:

Attributes are like properties or characteristics of individual objects in a software system. They represent specific information that is relevant to the system. For example, a FieldOfficer object may have attributes like badge number, name, and rank.

When identifying attributes, we consider only the ones that are important for the system to function properly. For example, a FieldOfficer's social security number may not be relevant to the emergency information system, so we don't include it as an attribute. Instead, we focus on attributes like badge number, which helps identify the FieldOfficer within the system.

Attributes have two main properties:

1. **Name:** Each attribute has a name that identifies it within an object. For example, the attribute "badgeNumber" represents the badge number of a FieldOfficer.
2. **Type:** Each attribute has a type that describes the legal values it can take. For example, the badge number attribute may

have a type of "integer" because it should only accept whole numbers.

To identify attributes, we can use some helpful rules called heuristics:

- Examine possessive phrases: Look for phrases like "belonging to" or "of" to identify attributes. For example, in the phrase "badge number of a FieldOfficer," "badge number" is an attribute.
- Represent stored state as an attribute: Consider attributes that represent the stored information or state of an object.
- Describe each attribute: Provide a clear description for each attribute, indicating what information it represents.

In simpler terms, attributes are like properties or characteristics of objects in a software system. They represent specific information that is important for the system to work correctly. Attributes have names and types. To identify attributes, we look for phrases that indicate possession or ownership, and we consider attributes that store important information. It's important to describe each attribute clearly to understand what it represents.

IDENTIFYING AGGREGATES

Let's simplify the explanation of aggregations and their part-whole relationship:

Aggregations represent a special type of association that shows a relationship between a whole and its parts. It's like a way to understand how smaller things come together to form a larger thing.

For example, imagine a FireStation. It consists of several components, such as FireFighters, FireEngines, Ambulances, and a LeadCar. Each of these components is part of the FireStation and contributes to its functioning.

Similarly, in a State, you can have Counties, and each County is composed of smaller units called Townships. These relationships help us understand how things are organized and connected.

In an analysis model, aggregations are represented as associations with a diamond-shaped symbol on the side of the whole (like the FireStation or State). This symbol indicates that there is a part-whole relationship between the objects involved.

In simpler terms, aggregations show how smaller things are connected to form a bigger thing. It's like putting puzzle pieces together to create a complete picture. In the example of a FireStation, there are different components like FireFighters and FireEngines that come together to make up the FireStation. The same idea applies to a State composed of Counties and Townships. Aggregations help us understand the relationships between these parts and the whole.

MODELING STATE DEPARTMENT BEHAVIOR OF INDIVIDUAL OBJECTS

Let's simplify the explanation of sequence diagrams and state machine diagrams:

Sequence diagrams show the behavior of the system from the perspective of a single use case. They help us understand the flow of actions and messages between different objects involved in that use case. It's like watching a step-by-step movie of how things happen in a specific situation.

On the other hand, state machine diagrams represent the behavior of a single object. They focus on showing how an object changes its state based on different events or actions. It's like looking at a map that guides us through the different states an object can be in and the transitions between those states.

By looking at the transitions in a state machine diagram, developers can discover new behaviors or actions that an object can perform. It helps us understand how an object reacts or changes based on specific events or triggers.

It's important to note that we don't need to create state machines for every class in the

system. Usually, state machines are more relevant for control objects that manage the flow of actions, less often for entity objects that represent real-world concepts, and almost never for boundary objects that interact with the users.

For example, let's take an Incident object in a system called FRIEND. This object can exist in different states: Active, Inactive, Closed, and Archived. Each state represents a specific situation related to an incident:

- **Active Incident:** It means there is an ongoing situation that requires a response, like a fire or a traffic accident.
- **Inactive Incident:** It means the situation has been handled, but there are still some reports or follow-up tasks to be completed, like finalizing damage estimates after a fire has been put out.
- **Closed Incident:** It means the situation has been handled and properly documented.
- **Archived Incident:** It's a closed incident whose documentation has been moved to a separate storage location.

In simpler terms, sequence diagrams show the step-by-step actions and messages between objects in a use case, while state machine diagrams show how a single object changes its state based on different events or actions. They help us understand the behavior of the system and individual objects. State machine diagrams are especially useful for understanding how objects transition between different states and react to specific triggers.